
20 MILLISECONDS

one Commodore 64 frame, cycle by cycle

the VIC-II explained step by step, with nothing to know in advance

Thirteen assembly instructions, ten experiments to run,
and every screenshot taken on a real machine:
from the red border to FLI.

July 2026 · English edition, October 2026

Companion volume to *Au cœur du métal* — *Commodore 64 & Ultimate 64* (in French)

Written with AI assistance (Claude) · cross-audited by Codex and Claude Opus 5
under the direction of **Johnny Piette**

Translated from the French original *20 millisecondes*, with AI assistance (Claude)
Licensed under CC BY-SA 4.0

Contents

Before you begin	4
Chapter 0 — The Machine from Above	5
The promise	5
A tour of the workshop	5
The idea that drives the whole book	5
An address can be a lever	6
Mission 0 — set up the workshop, run the trailer	6
Next chapter	8
Chapter 1 — 6502 Survival Kit	9
The question	9
The first four	9
The experiment: red border	9
The stub, explained once and for all	11
As fast as possible	11
Next chapter	12
Chapter 2 — The Frame: Seeing Time	13
The question	13
What the beam does, in twenty milliseconds	13
The locker that tells the time	13
Watching, rather than being notified	13
The experiment	14
The detail that announces the next chapter	15
Next chapter	15
Chapter 3 — The Line: 63 Beats	16
The question	16
The machine's unit of measure	16
What each instruction costs	16
A color planned for each line	16
The experiment	17
The geometry of the tears	18
Next chapter	18
Chapter 4 — The Bad Line: When the Artist Requisitions the Corridor	19
The question	19
Two instructions to build a stopwatch	19
The experiment	19
What happened in the corridor	20
The counter-test	20
Patience, and a prediction verified to the pixel	21
Next chapter	23
Chapter 5 — The Cells: Where Characters Come From	24
The question	24
The experiment — a thousand lockers	24
The old hand's elegance: 1000 lockers in four passes of 256	26
Screen codes are NOT ASCII codes	26
Color RAM: a thousand more lockers, and a trap	26
These thousand bytes are the Bad Line's loot	27
Second experiment — the screen is just an address	27
Where to put the second matrix? (a decision, not a recipe)	27
One letter per cell, at chosen places	27
The explanation — $132 = 128 + 4$	29

Next chapter	30
Chapter 6 — Sprites: Eight Free Objects	31
The question	31
The drawing: twenty-one lines of 0s and 1s	31
The experiment — one sprite, motionless, in the center	31
The explanation — why 64?	34
The experiment — all eight, in a row	34
The experiment — the eight thieves	37
The explanation — what a sprite really costs	40
Next chapter	41
Chapter 7 — The Hidden Counters, and the Screen That Falls	42
The question	42
The VIC's bookmark	42
The offset trap, and why you must aim two lines ahead	42
The experiment	42
What you have just gained	44
Next chapter	44
Chapter 8 — Opening the Border	45
The question	45
The border is not a zone: it is a tap	45
Two values, one switch	45
Changing one switch without touching the others	45
The experiment	46
The ghost byte	48
Next chapter	49
Chapter 9 — The Grand Finale: All the Colors at Once	50
The question	50
What graphics mode really is	50
The idea: a Bad Line on every line	50
Three attempts, and what each one teaches	50
Where the VIC goes to fetch all this	52
The experiment	52
The scar	54
Why 24 rows, and not 25	54
What you know how to do	55
Next chapter	55
Chapter 10 — The Same Machine in 2026	56
The question	56
A machine described, not imitated	56
Turbo: sixty-four times faster, and still twenty milliseconds	56
The freight elevator: the REU	57
The gateway	57
Coming full circle	58
Appendix — Three Pages to Keep at Hand	59
The Sixteen Colors	59
The Eight Switches of \$d011	59
The 63 Cycles of a Line	59
The Assembler Notations	60
The Thirteen Instructions, and What They Cost	60
The Screen Codes Used in This Book	61
What All the Programs Assume	61
How to Exit a Program	61
Sources	62
The Companion Volume	62
The VIC-II, and Time	62

The Processor	62
The Chips This Book Left Alone	63
The 2026 Machine	63
The Original Manual	63

Before you begin

This book asks only one thing of you: that you have programmed a little before. Anything, in any language — if the words *loop* and *variable* don't scare you, you have everything you need. It requires **neither** knowing the Commodore 64, **nor** knowing assembly language. You will learn both here, along the way, and only as far as necessary.

The contract is the same in every chapter: a **question** you can see on the screen, an **experiment** — a short, complete program that you type in and run —, what you **observe**, then the **explanation**. Never the other way around. Theory only arrives after your eyes have seen the phenomenon.

One last thing, for the road: the book ends with a **three-page appendix** that gathers what you keep looking up when you program — the sixteen colors, the eight switches of the most important register, the map of the 63 cycles of a line, and the assembler's notations. Nothing obliges you to read it in advance; just know that it is there.

And a promise: **every screen image in this book is real**. Each one was produced by the listing printed just above it, run on a real machine, and captured as is. None was drawn, simulated or retouched.

Chapter 0 — The Machine from Above

The promise

Every twenty milliseconds, your Commodore 64 draws a complete picture.

Not “vaguely refreshes the display”: it draws it, entirely, dot by dot, line by line — 312 lines, from top to bottom, the way the electron beam of a CRT television once scanned the screen. Fifty times a second, without a single exception, without ever being late. While your program runs, while it crashes, while you do nothing at all: the picture, for its part, gets delivered. Every twenty milliseconds.

This book tells what happens during **one single** one of these frames. Twenty milliseconds, slowed down until every microsecond becomes visible, then every **cycle** — the elementary beat of the machine, a little over a millionth of a second. One frame is exactly 19,656 beats. By the end of the book, you will know how to read those beats: what each instruction costs, who they belong to, which ones will be stolen from you and when — and above all, you will know how to slip into the gaps and make the machine do things its designers never planned.

It’s a journey you have to earn, but it pays cash: starting with this chapter, you will run a program. In the next chapter, you will write one.

A tour of the workshop

Open up the machine — mentally, that will do. Inside, a workshop. Small, dense, and organized around a single constraint from which everything else follows.

The storeroom: 64 kilobytes of memory. Imagine 65,536 lockers in a row, numbered from 0 to 65535, each holding one byte — a number between 0 and 255. That’s all. The text of your program, the pictures, the sounds, your variables: everything that exists in this machine lives in these lockers.

The single corridor: the bus. To read or write a locker, you have to take the corridor that leads to the storeroom. And this corridor has only one lane: *one* access at a time, about one per microsecond. Remember this corridor. Half of this book — and the finest tricks of the Commodore 64 — are stories about the corridor.

The two craftsmen. Two workers labor in the workshop, and they don’t have the same relationship with time:

- The **6510**, the processor. It executes your instructions, one by one, conscientiously. It calculates, compares, moves bytes. It has a rare quality: it is *patient*. If you make it wait, it waits.
- The **VIC-II**, the video chip. The workshop’s artist — and an artist under a merciless contract: it must deliver a frame every twenty milliseconds. Not “as soon as possible”: *on time*. The beam sweeping the screen waits for no one; if the VIC missed its appointment, the picture would tear apart on the spot. It cannot afford to be patient.

The supporting cast. Two other chips deserve a nod: the **SID**, the musician (three voices, a legendary temperament), and the two **CIAs**, the doorkeepers — keyboard, joysticks, clocks. This book will leave them alone: our business is the picture.

The idea that drives the whole book

Two craftsmen, a single corridor: they have to share. And the sharing isn’t negotiated amicably — it follows from the constraint: *the VIC can’t be late, the processor can wait*.

So the machine cuts every microsecond into two halves: one for the VIC, one for the 6510. Each gets its half-microsecond, each gets its accesses to the storeroom, and in the normal case nobody steps on anybody’s toes. You will discover in chapter 4 what happens when the VIC’s half is no longer enough for it — it’s one of the best-buried secrets of the machine, and the source of its most famous oddities.

For now, remember the sentence that sums up this balance of power, because the whole book follows from it:

In a Commodore 64, time belongs to the video chip. The processor works in the time the VIC leaves it.

An address can be a lever

One last workshop secret before we move on to the practical work. Among the 65,536 lockers, a few dozen **are not memory**. They are wired directly to the chips: writing into them means *pulling a lever*. Locker 53280 (the regulars write its number in hexadecimal: \$d020) is wired to the VIC — write a number from 0 to 15 into it, and the screen border changes color, immediately. You don't need to understand assembly to find that remarkable: *changing a color means writing into a memory cell*.

All the programming in this book boils down to that idea: knowing **what to write, into which locker — and at what moment**. The first two points are chapter 1. The third is the rest of the book.

Mission 0 — set up the workshop, run the trailer

You need two tools, free and available on every system:

- **VICE**, the emulator (the x64sc program, the most faithful emulation) — vice-emu.sourceforge.io. If you own a real machine — a period C64, or one of its hardware reissues (Ultimate 64, C64 Ultimate) — even better: all of this book was tested on real metal.
- **ACME**, the assembler — the translator that turns a source text into an executable program. Only one command to know: `acme file.a`. Take **version 0.95 or later** (this book was assembled with 0.97): earlier versions don't know the form of loop used in chapters 7 and 9.

All the programs in this book are provided alongside it, one folder per chapter (ch00/, ch01/...), with the screenshot that each one actually produced. You can therefore type them in — it's formative — or start from the file. Here is the book's trailer:

```

;-----
; 20 MILLISECONDES – chapter 0: the trailer
;
; You won't understand this listing yet. That's normal.
; It's the destination: by the end of the book, it will hold
; no more secrets for you – and you'll be able to do much better.
;
; Assemble :   acme teaser.a
; Run :       LOAD"TEASER",8,1 then RUN
;-----
!to "teaser.prg", cbm

* = $0801                ; BASIC stub: 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence: nobody interrupts us any more

boucle ldx $d012        ; which LINE is the beam on?
      lda couleurs,x    ; the color planned for this line...
      sta $d020        ; ...painted on the border
      sta $d021        ; ...and on the background
      jmp boucle       ; forever

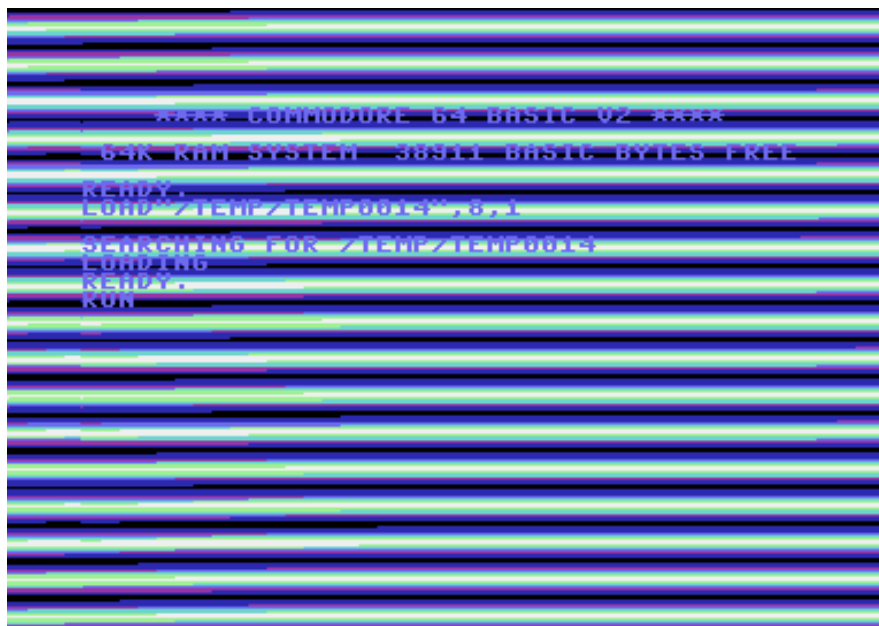
; a wave of 16 colors, dark -> light -> dark
!macro vague { !byte 0,6,6,4,14,3,13,1,1,13,3,14,4,6,6,0 }

couleurs                ; 256 lines = 16 waves
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague

```

Assemble it (acme teaser.a), run the resulting teaser.prg (drag it onto the VICE window, or LOAD"TEASER",8,1 then RUN on a real machine).

What you observe:



Real capture (C64 Ultimate, native video output). The BASIC startup text is still there — waves of color run across

it, from the top to the bottom of the screen, border included.

Look closely: the color changes **on every scan line**. Not on every character, not on every “cell” — on every line of the beam, 312 times per frame, 50 times a second. No “normal” language can do that: the C64’s BASIC barely executes a few dozen instructions while an *entire* frame is drawn. Here, the program converses with the beam *while* it scans — and it takes only six lines.

You don’t yet know how to read `$d012`, nor why the `couleurs` table has 256 entries, nor what exactly `sei` forbids. That’s the whole promise: **chapter 2** for the beam and `$d012`, **chapter 3** to understand — down to the microsecond — why the waves slightly “lean” at the edges of the picture you have before your eyes. Nothing in this capture is a flaw: all of it is already a lesson in timing.

Under the hood — for readers in a hurry to check: the sharing of the corridor in half-microseconds (the $\phi 0/\phi 2$, AEC and BA signals) is specified in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 1, §1 — the reference volume of which this book is the guided tour: a synthesis of the original sources, austere and made to be consulted, not studied. Every chapter will end with a cross-reference of this kind: here, the phenomena; there, the exact rules and their sources.

Next chapter

You have run someone else’s program. In chapter 1, you will write your own: thirteen assembly instructions — not one more in this whole book — and the first lever pulled by your own hands.

Chapter 1 — 6502 Survival Kit

The question

You have run someone else's program. How do you write your own?

You will have to speak to the machine in its own language — and this is where books usually lose half their readers, in a thirty-page chapter on addressing modes. We are going to do it differently. This entire book uses only **thirteen instructions**. Not thirteen to start with: thirteen in all, up to the last page, FLI included. You will learn **four** in this chapter, and you will already have enough to write a program that works.

The idea you have to give up first: assembly is not a language to learn, with its grammar and its idioms. It is a **translator**, and a translator without imagination: every line you write becomes exactly one order for the processor, no more, no less. `lda #2` becomes two bytes. There is nothing underneath. It is precisely this stripped-down quality that will give you control of time, in chapter 3.

The first four

The processor has three little cells of its own — one byte each — in which it holds what it is working on right now. They are called **registers**: A, X and Y. A is the working register, the accumulator; X and Y will wait for chapters 3 and 5. Everything the 6510 does consists of loading a byte into a register, fiddling with it, then dropping it somewhere.

New instruction — LDA (“LoaD A”): puts a byte in register A. `lda #2` loads **the value 2**. `lda $d020` loads **the contents of locker \$d020**. The # changes everything: with it a value, without it an address. It is the only really costly confusion in 6502 assembly — if a program behaves strangely, it's the first place to look.

New instruction — STA (“STore A”): drops the contents of A into a locker. `sta $d020` writes into locker 53280. There is no `sta #2`: you don't store anything “into a value”, the idea makes no sense.

A word or two on this \$d020 notation. The \$ announces a number in **hexadecimal**, that is, counted in bundles of sixteen instead of ten. It's not an insiders' affectation: the machine is built on bundles of bits, and in hexadecimal the boundaries come out round. \$d020, \$d021, \$d022... visibly follow one another, whereas 53280, 53281, 53282 evoke nothing. You have nothing to convert in your head: read \$d020 as a proper name, the name of the border's locker.

New instruction — JMP (“JuMP”): continues execution elsewhere. `jmp boucle` resumes at the spot you have christened boucle. It's the machine's goto — and here, nobody will hold it against you: it's our only tool for repetition until the next chapter, which will add a finer one.

New instruction — RTS (“ReTurn from Subroutine”): hands control back to whoever called you. Since it's BASIC that launched our program (with its SYS), `rts` takes us back to READY.

The experiment: red border

Here is the entire program. Don't be intimidated by the dozen bytes at the start: I explain them right afterwards, once and for all.

```

;-----
; 20 MILLISECONDES — chapter 1: your first program
;
; Three instructions. The border turns red, and BASIC
; takes over again as if nothing had happened.
;
; Assemble :   acme bordure.a
; Run :       LOAD"BORDURE",8,1 then RUN
;-----
!to "bordure.prg", cbm

; --- the stub: one line of BASIC, stored by hand in lockers ---
* = $0801
!byte $0b,$08           ; where the NEXT line begins ($080b)
!byte $0a,$00           ; the line number: 10
!byte $9e               ; the word SYS, stored as a single byte
!byte $32,$30,$36,$34   ; the characters "2 0 6 4"
!byte $00               ; end of the line
!byte $00,$00           ; end of the BASIC program

; --- our program, at address 2064 ($0810) ---
* = $0810
    lda #2              ; A <- the VALUE 2 (red)
    sta $d020           ; ...dropped into the border's lever-locker
    rts                 ; and we hand control back to BASIC

```

Two lines of this listing are not instructions but orders given to the assembler: `* = $0801` and `* = $0810` mean “store what follows starting at this address” — the star stands for the current address, it is not a multiplication. All notations of this kind are gathered in the appendix.

Three useful lines. Assemble it (`acme bordure.a`), run the resulting `bordure.prg`.

What you observe:

Real capture. The odd path in the load line is that of our test machine, which receives programs over the network rather than from a floppy — on your machine, it will be the name of your file.

Two things deserve your attention. First, the `READY.`: our program is **finished**, BASIC has taken over again, you can type `PRINT 2+2` as if nothing had happened. Second — and this is the lesson of the chapter — **the border stayed red**. Nobody keeps it red. No loop repaints it fifty times a second. We dropped a 2 into a locker, and that 2 stays there: the VIC rereads it by itself, for every pixel of border in every frame, until somebody writes something else.

(The sixteen colors of the Commodore 64 and their numbers are gathered in the appendix, at the end of the book: it's the page you will consult most often.)

That is the very nature of a **lever-locker**: you don't give it orders, you *set* it. Remember this, because starting in chapter 7 we will do exactly the opposite — rewriting a register over and over, at chosen instants, to fool the VIC. But for that, we will have to know *when*.

The stub, explained once and for all

These dozen bytes at the start will follow you through every program in the book. It is there because you can't launch machine code directly from the C64's home screen: you have to go through BASIC. So we build it a tiny program — 10 SYS 2064 — not by typing it, but by storing by hand the bytes that represent it.

For a BASIC program, in this machine, is nothing but bytes in lockers, exactly like everything else. The line 10 SYS 2064 is written like this: the address of the next line (two bytes), the line number 10 (two bytes), a \$9e byte that *is* the word SYS (BASIC stores its keywords one byte each — that's why a BASIC program takes up so little room), then the four characters "2064", then zeros to say "it's over". BASIC's SYS jumps to address 2064, where our three instructions are waiting. 2064 is \$0810: right after the stub.

You will never have to think about it again. Copy these twelve bytes at the top of every program, and consider them the customary "hello" addressed to BASIC.

As fast as possible

One last experiment, and it will pose the question of the whole book. What happens if we change the border color **as fast as the machine is capable of**?

```
!to "stroboscope.prg", cbm

* = $0801                ; the chapter's stub: 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
boucle lda #0            ; black
      sta $d020
      lda #1             ; white
      sta $d020
      jmp boucle         ; and again, never stopping
```

Your intuition says: the border will flicker — probably too fast for the eye, so it will look gray. Run it.

What you observe:



*Real capture. The border is neither black, nor white, nor gray: it is **striped**.*

Stripes. Not a flicker, not a uniform gray: black and white bands perfectly drawn, motionless. Look at them closely, they contain everything this book has to say.

What you see is a **race** between two things moving at roughly the same speed. Our loop is so short that it repaints the border **eight times while a single screen line is drawn** — black, white, black, white, four times in a row. Each line is therefore cut into eight alternating slices, and since the loop and the scan never fall perfectly in phase, the cutting shifts a little with each line: hence these stripes. (You will be able to

recount that “eight” yourself in chapter 3.)

You have just met the main character of the book: **the beam**. It does not wait for your program, it does not speed up for it, it advances at its own pace — and the whole craft of the Commodore 64 consists in writing at the right place *as it passes*.

We now know how to write into a locker. Exactly one thing is missing: knowing **where the beam is**.

Under the hood — the VIC’s 47 lever-lockers, their addresses and the role of each of their bits, are given in a single table in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 1, §2; the palette of the sixteen colors and the way they are made (by mixing signals, with no red-green-blue at all) in §3.

Next chapter

The C64 has a locker that answers this question, permanently, for free. We are going to read it — and our random stripes will become a band of color perfectly motionless, placed exactly where we have decided.

Chapter 2 — The Frame: Seeing Time

The question

Our stripes from chapter 1 were pretty, but we had no say in them. We want to decide: a band of color, here, at this height, standing still. For that, we need to know where the beam is.

What the beam does, in twenty milliseconds

Here is the complete journey, and there is nothing more you need to know about it for this chapter. The beam starts at the top left, crosses the screen to the right, returns to the left one line lower, and starts again. On PAL — the European machines — it draws **312 lines** this way, then climbs back to the very top and sets off again for the next frame. Fifty times per second.

Not every line is visible: the first and the last fall outside the panel, a legacy of the TVs that needed this dead time to bring their beam back to the top. On the lines that remain, there is a strip of border on the left, one on the right, and in the middle the display window: 320 pixels wide, 200 tall.

What matters to us is that this journey is **perfectly regular**. The beam does not jump, does not speed up, never misses its appointment. It is the most reliable clock in the machine — and the C64 lets us consult it.

The locker that tells the time

The \$d012 locker — the line counter. At any instant, it holds the number of the line the beam is currently drawing: a value that changes all by itself, **about 15,600 times per second** (312 lines, 50 frames). You can also write to it — that is used to program an interrupt, which this book does not use; so we will be content to **read** it.

A word of honesty right away: a locker holds only one byte, so 255 at most, and there are 312 lines. The full number needs a ninth bit, which lives elsewhere (in \$d011). Past line 255, \$d012 therefore starts again from zero for the end of the frame. We will wisely stay below 255 in this chapter — that is, within almost the whole visible screen.

Watching, rather than being notified

Our strategy will be the simplest in the world: read \$d012 in a loop until it shows the value we expect. We call that **watching** (English speakers say *polling*). The program does nothing but watch the beam go by, and acts the moment it arrives.

This is not the professionals' method: the C64 can *notify* the processor when a given line is reached, which leaves all the time in between free for something else. But that mechanism requires installing an interrupt handler, saving registers, understanding detours that have nothing to do with the VIC. This book made a radical choice: **everything, up to the FLI of chapter 9, will be done by watching**. You will see that it is more than enough — and that it keeps the program readable at a single glance.

We do, however, have to silence someone. Sixty times a second, the C64's operating system interrupts whatever is running to scan the keyboard, blink the cursor, advance its internal clock. More than two hundred microseconds stolen — three and a half screen lines — at the worst moment: our watch would miss its line.

New instruction — SEI ("SEt Interrupt disable"): closes the door. After sei, nobody interrupts our program anymore. The keyboard stops responding (RUN/STOP included) — to get control back, you will need RUN/STOP + RESTORE, which goes through another door, or power off. That is the price of silence, and every program in this book pays it.

New instruction — CMP ("CoMPare"): compares register A with a value, without modifying anything. cmp #\$80 asks: "Is A equal to 128?"

New instruction — BNE (“Branch if Not Equal”): jumps if the last result was **not** zero. After a `cmp`, “zero” means “equal”: `bne` therefore jumps when the comparison found a difference. This is our “while”: `cmp #$80` followed by `bne haut` means “while it is not 128, go back to watching.”

The experiment

```
!to "bande.prg", cbm

* = $0801                ; 10 SYS 2064 (the BASIC stub from chapter 1)
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                  ; the system will no longer interrupt us

; --- watch for line $80 (128), in the middle of the screen ---
haut    lda $d012         ; where is the beam, RIGHT NOW?
        cmp #$80         ; has it reached line 128?
        bne haut         ; no: keep watching
        lda #$0a         ; yes: light red
        sta $d020        ; border
        sta $d021        ; and background

; --- watch for line $a0 (160), 32 lines lower ---
bas     lda $d012
        cmp #$a0
        bne bas
        lda #$0e         ; light blue: the original border
        sta $d020
        lda #$06         ; blue: the original background
        sta $d021

        jmp haut         ; and start over, frame after frame
```

Fifteen lines in all, with no new instruction except the three from this chapter. Note also the pattern to remember, which we will meet everywhere in this book: **load \$d012, compare, loop if it is not yet time.**

What you observe:



Real capture. The band crosses the border as well as the display window: we are painting the backdrop, not the characters — the BASIC text goes on living its life on top.

A band. Motionless. Thirty-two lines tall, placed where we asked. We did not draw a rectangle: we changed the color of the backdrop **during two short windows of time**, and the beam, passing by, did the rest. The rectangle exists nowhere in memory.

Take a moment to measure what just happened. The C64 has no function for drawing a band of color across the border — that is even precisely what its hardware *cannot do*. We got it by writing twice into a locker, at the right moment. That is C64 programming: the machine knows no shapes, it knows moments.

The detail that announces the next chapter

Take a closer look at the capture, at the left edge of the band, where the red **ends**. The transition is not perfectly clean: on the first line below the band, the border has already gone back to its light blue, while the background stays red for about twenty more pixels.

This is neither a flaw in the capture nor a flaw in your machine. When our `lda $d012` finally reads 160, the beam is **already drawing** line 160: it moved on while we were comparing, while we were loading the color, while we were writing it. And we write the border first and the background after — six cycles later, which amounts to forty-eight pixels (the next chapter gives the conversion).

In other words: we now know how to aim at a line. We do not yet know how to aim at a **place within** the line. And that is where all the tricks of the Commodore 64 hide.

Under the hood — the number of lines and cycles for each variant of the chip (PAL, NTSC, and their revisions), the exact dimensions of the display window, and the ninth bit of the line counter are in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 1, §4; the interrupt mechanism that replaces our watching is in §14.

Next chapter

We are going to zoom in one notch: inside a single line. You will discover that it lasts exactly 63 beats, that each of our instructions consumes a number of them known in advance — and you will finally be able to read, line by line, the trailer from chapter 0.

Chapter 3 — The Line: 63 Beats

The question

In the previous chapter, the bottom of our band left a misaligned seam of a few dozen pixels. We said “the beam moves on while we work.” How much, exactly?

It is the most profitable question in the whole book. Answering it means zooming in one notch: until now we were counting lines, now we will count inside a line.

The machine’s unit of measure

A screen line, on a European (PAL) C64, lasts exactly **63 cycles**.

The cycle is the processor’s beat: 1.015 millionths of a second, to be exact — let’s call it a microsecond and not come back to it. It is also, and this is where everything is decided, the time the beam takes to cross exactly **eight pixels**. One cycle, eight pixels: this equivalence is the most useful rule of three on the Commodore 64. It says that time and space are the same thing seen from two angles — a delay of ten cycles reads on screen as a shift of eighty pixels.

Let’s do the sums for our frame: 63 cycles per line, 312 lines, that makes 19,656 cycles per frame. You recognize the number: these are the 19,656 beats announced in chapter 0. At a little over a microsecond each, they make up our twenty milliseconds ($19,656 \times 1.015 \mu\text{s} = 19.95 \text{ ms}$: that is where the title of this book comes from). The loop is closed: we finally have both ends of the scale, from the beat to the frame.

What each instruction costs

Here is the price list for the instructions we know. These numbers are not invented and cannot be guessed: they are wired into the processor, published, and verifiable to the cycle.

Instruction	What it does	Cost
lda #2	load a value	2 cycles
lda \$d012	load the contents of a locker	4 cycles
lda couleurs,x	load cell no. X of a table	4 cycles
sta \$d020	store into a locker	4 cycles
cmp #\$80	compare with a value	2 cycles
bne boucle	jump if different	2 cycles if not taken, 3 if taken
jmp boucle	jump	3 cycles
sei	close the door on interrupts	2 cycles

Let’s take the watch loop from chapter 2 with this grid in hand: lda \$d012 (4), cmp #\$80 (2) and bne (3 when it loops) make **9 cycles per turn of the watch**. Nine cycles is 72 pixels.

That explains the botched seam: when our loop finally notices the arrival of line 160, the beam may already have advanced nine cycles into that line. And between writing the border and writing the background, it advances another 6 cycles — the 2 of the lda #\$06 and the 4 of the sta: forty-eight pixels. Our band could not switch right at the left edge. It switched where the machine was.

None of this is a flaw. It is the rule of the game, and it is *known in advance*: that is exactly why it can be worked around. Chapters 7 to 9 will do nothing but count cycles to land in the right place.

A color planned for each line

We have what we need to understand the trailer from chapter 0. One tool is missing, and only one: a second register to carry the line number, and the ability to use it as an index into a table.

New register and new instruction — LDX (“LoaD X”): like lda, but for the X register. ldx \$d012 puts the current line number into X.

New mode — indexed: `lda couleurs,x`: loads not the contents of `couleurs`, but those of cell `X` of the table that starts at `couleurs`. If `X` is 128, it reads the 129th byte of the table. One instruction, four cycles, and you have the indexed array of ordinary languages.

These two make up the **signature pattern** of this book:

```
ldx $d012          ; where is the beam?
lda couleurs,x     ; what had I planned for this line?
```

Remember it: we will meet it again in chapter 7 to make the screen fall, and in chapter 9 for the FLI. It is the most economical way there is to make something vary line by line — because **all the calculation was done in advance**, by the assembler, when the program was built. During display, all that is left is to read.

The experiment

```
!to "degrade.prg", cbm

* = $0801          ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei           ; silence

boucle ldx $d012    ; 4 cycles: X <- the current line
      lda couleurs,x ; 4 cycles: the color planned for THIS line
      sta $d020      ; 4 cycles: border
      sta $d021      ; 4 cycles: background
      jmp boucle     ; 3 cycles: and start over
                      ; total: 19 cycles per turn of the loop

; --- the table: one color per line ---
; !align guarantees it starts at the beginning of a 256-byte page:
; without it, "lda couleurs,x" would sometimes cost 5 cycles instead of 4,
; and our loop would no longer beat regularly.
!align 255, 0

; a wave of 16 shades: dark -> light -> dark
!macro vague { !byte 0,6,6,4,14,3,13,1,1,13,3,14,4,6,6,0 }

couleurs          ; 16 waves = 256 lines
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
```

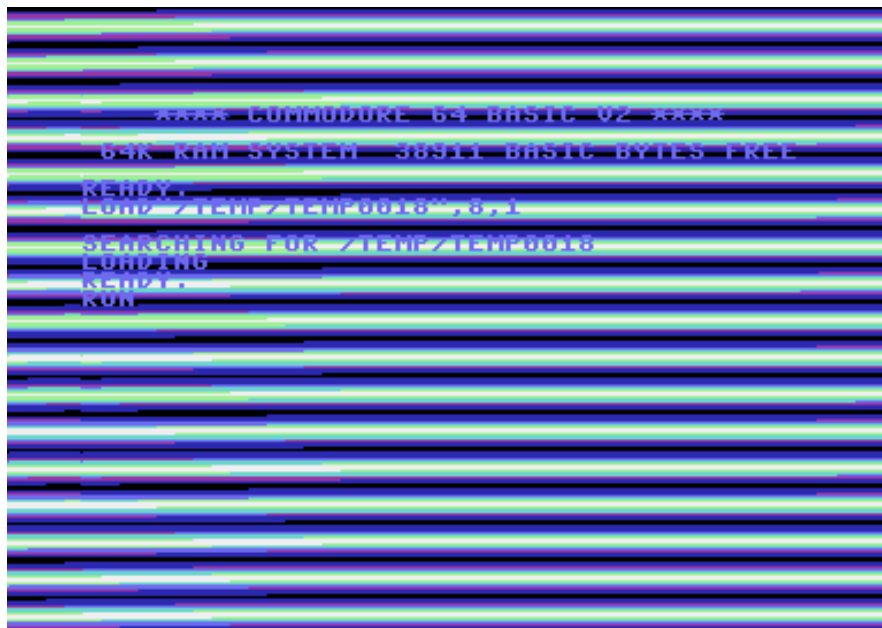
A couple of words on the last lines, which are not instructions. `!macro` and `+vague` are conveniences of the assembler: we describe a wave of sixteen shades, we ask for it sixteen times, and the assembler writes the 256 bytes.

And these sixteen shades are not random: 0, 6, 6, 4, 14, 3, 13, 1 then the same in reverse. They are color numbers — black, blue, pink, light blue, cyan, light green, white — arranged **from darkest to lightest**, then back down. Two properties follow: the rise is smooth, because we follow the palette's brightness scale and not the order of the numbers; and the wave closes back on itself, because it ends as it began. The sixteen repetitions therefore join up seamlessly. The processor will know nothing of this: it will simply find a ready-made table. **Everything that can be computed at build time costs nothing at display time** — that is the principle that will make chapter 9 possible.

As for `!align`, it deserves its comment in the listing. Our table is exactly 256 bytes; if it starts at the beginning of a 256 page, then `lda couleurs,x` stays within that page whatever the value of `X`. Otherwise, for

high values of X , the processor has to fix up the address and the instruction costs **5 cycles instead of 4**. A loop that sometimes beats 19 cycles and sometimes 20 is a loop that drifts. We have just paid one line of assembler to buy regularity.

What you observe:



Real capture. Compare with the trailer from chapter 0: these are the same six instructions, give or take one line — the one in chapter 0 had no `!align`, and that is precisely why its waves were a bit messier than these.

The geometry of the tears

It remains to explain what you are really seeing — because the waves are not clean bands, they have stair-stepped edges and glitches.

Our loop lasts 19 cycles. A line lasts 63. So the loop runs **three times per line**, more or less: three times 19 is 57, leaving 6 cycles over. “More or less” is the important phrase — 63 is not a multiple of 19, and that remainder of 6 cycles accumulates from one line to the next. The moment when the loop catches the line change therefore drifts, line after line: it is this slippage that draws the staircase of the edges.

And the three passes within the same line? They all three read the same line number, so they write the same color three times: invisible. But the one that straddles the line change writes the old color at the start of the new line — hence, at the left edge of the transition lines, those little segments that still carry the color of the previous line.

Note the conclusion, which is the program for the next six chapters: **our loop is a little slower than it should be, and above all, it is not aligned with the line**. To get perfectly crisp bands, we would need to know *exactly* how many cycles elapse between two writes, and to resynchronize on each start of line. We will learn both.

Under the hood — the cycle cost of each 6510 instruction, addressing mode by addressing mode and step by step, is in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 3 (after the *64doc* document): that is where the eight numbers in the table above are verified, along with the extra cycle of page crossings.

Next chapter

But there is an obstacle, and a sizable one: the processor is not alone in the machine. Once every eight lines, the VIC takes the corridor from it, and our so carefully timed loop suddenly loses some forty cycles. It has a name, it shows to the naked eye, and it is the heart of the Commodore 64.

Chapter 4 — The Bad Line: When the Artist Requisitions the Corridor

The question

We know what each instruction costs. So we can, in principle, predict exactly where the beam will be at the end of any sequence of instructions.

In principle. This chapter is where the machine makes liars of us — and it is worth the trouble, because the lie is regular, measurable, and has a name.

Two instructions to build a stopwatch

New instruction — INC (“INCrement”): adds 1 to the contents of a locker, in place, without going through the A register. `inc $d021` steps the background color to the next one. It costs 6 cycles: read, add, write back.

With `inc` and `jmp`, we can write the shortest and most revealing program in the book. Two instructions: increment the background color, start over.

The count is immediate: 6 cycles for the `inc`, 3 for the `jmp`, so **9 cycles per color change**. And 9 cycles is 72 pixels. Every stripe you are about to see on the screen *is* a measurement: it says “here, nine beats went by.” The screen becomes a chronograph, and the beam its stylus.

Look again at the number 63, the length of a line. **63 divided by 9 is exactly 7**. In other words, if nothing disturbs our loop, every line will contain exactly seven stripes, and every line will be identical to the one before: the stripes will stack into **perfectly vertical columns**. Any distortion we see will therefore, without a doubt, be stolen time.

The experiment

```
!to "badline.prg", cbm

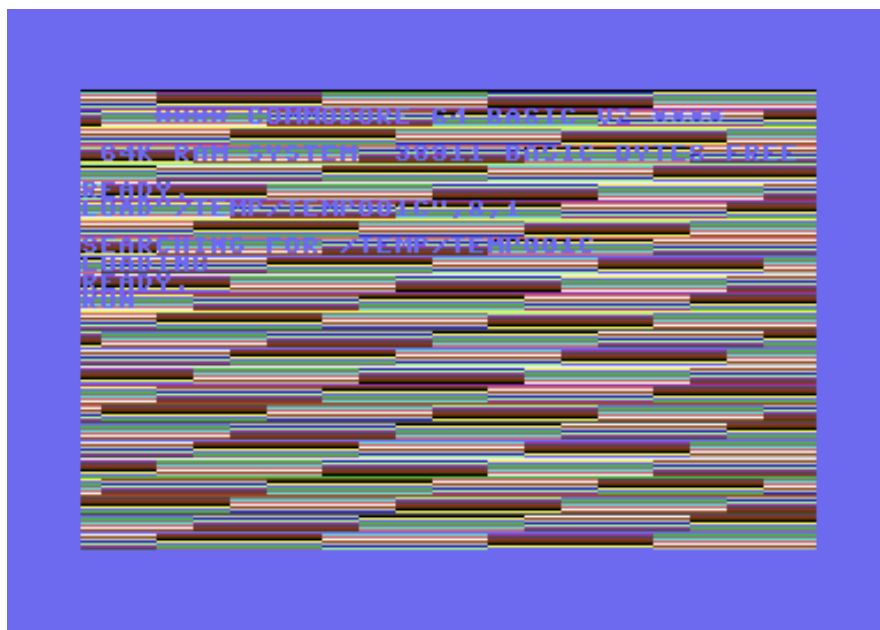
* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence

boucle inc $d021          ; 6 cycles: BACKGROUND color + 1
      jmp boucle         ; 3 cycles -> 9 cycles per stripe
                        ; 63 / 9 = 7 stripes per line, exactly
```

We paint the **background** and not the border, for a precise reason: the background is visible in the display window, which is exactly where the VIC is at work. That is where we have to set up our stopwatch if we want to watch it get robbed.

What you observe:



Real capture. The columns are there... and one line in eight cuts across them in a single block of uniform color.

There's the anomaly. One line in eight, our stopwatch **stops**: instead of its seven color changes, the line shows almost none. It stays one single shade across its whole width.

This is not an impression — and here I switch to “I,” because what follows is no longer a shared line of reasoning but a check that I made, with the machine switched on, and which you should be able to doubt. So I counted the color changes, line by line, in the capture above: ordinary lines show 4 to 5 (some of the stripes fall outside the visible frame), and one line in eight shows **0.29 on average**. One in eight, our program did not have time to do anything at all.

What happened in the corridor

Remember the workshop from chapter 0: a single corridor to the storeroom, split into two halves — one for the VIC, one for the processor. That arrangement works for pixels: the VIC reads eight pixels at a time, which is more than enough for it.

But every eight lines, the VIC has to do something else. It has to fetch the **next line of text**: which characters to display, and in what color. Forty characters, forty colors. And there, its half-microsecond per cycle is not enough, not even close.

So it does what an artist under a merciless contract is entitled to do: it **raises its hand**. A signal (electronics people call it BA, *Bus Available*) warns the processor three cycles ahead. The 6510 cleanly finishes the move in progress, then freezes — not “it slows down”: it stops, completely. The VIC then takes the **whole** corridor, for itself alone, for **40 to 43 cycles**. Out of the 63 that the line lasts.

This is what C64 programmers call a **Bad Line** — a “bad line.” The name is unfair, by the way: there is nothing bad about it, it is the price of display. Without it, the screen would be empty.

When does it occur? Every eight lines, since a character is eight pixels tall: once the line of text has been read, the VIC has enough to last eight scan lines. More precisely, it occurs when the **last three bits of the line number** match the **vertical scroll value** — a setting with eight positions, stored in the locker \$d011, which normally serves to slide the picture vertically by zero to seven pixels. Its default value is 3. And indeed, in the capture above, the knocked-out lines are the ones whose number ends in 3.

Hold on to that last sentence. The fact that the Bad Line depends on a register that **we can write** is the most consequential secret of the whole machine: it is the gateway to chapters 7 and 9.

The counter-test

An explanation that cannot be refuted is worth nothing. If Bad Lines exist because the VIC has to read characters, then **by switching the display off, they must disappear**.

The locker \$d011 happens to contain a “screen on” switch. Its usual value is \$1b; by writing \$0b, we switch the display off. (This locker holds eight independent switches and will come back in chapters 7, 8 and 9:

the appendix gives them one by one, with the five values that this book writes into it.) — the VIC has nothing left to read, nothing left to display, and the whole screen becomes border.

```
!to "eteint.prg", cbm

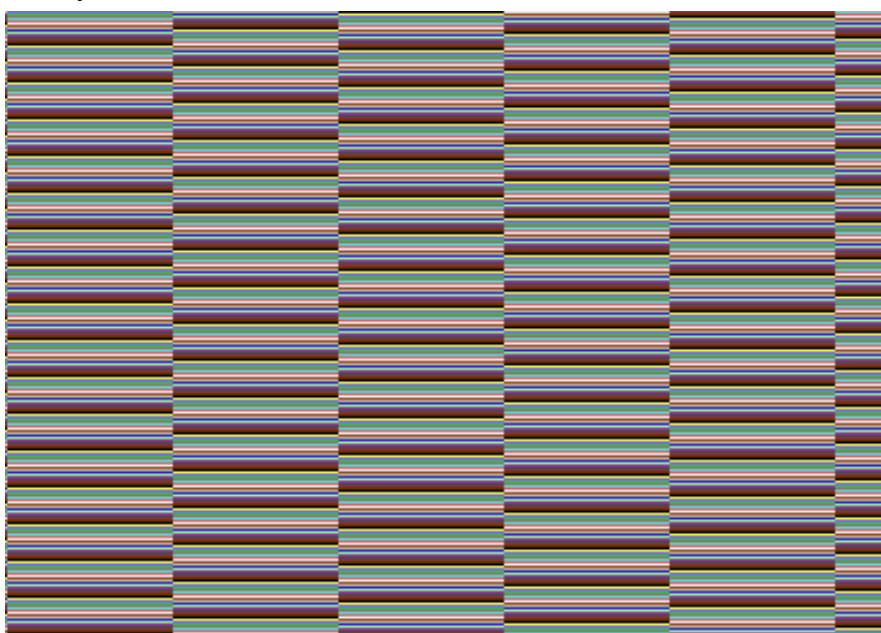
* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence
    lda #$0b           ; like $1b, but without the
    sta $d011          ; "screen on" switch

boucle inc $d020        ; 6 cycles
      jmp boucle        ; 3 cycles -> 9 cycles per stripe
```

A detail that matters: we go back to painting the **border** and not the background. This is no distraction — with the display off, there is no window at all any more: the entire screen *is* border. Our stopwatch therefore gets the whole surface.

What you observe:



Real capture, screen off. Seven stripes per line — six visible boundaries, the seventh falling outside the captured frame — aligned to perfection over all 272 lines: not a single Bad Line any more.

Impeccable. I checked on the machine: **exactly six color changes per line** in the capture window, on **every** line without exception, and an offset of **zero pixels** from one line to the next. Our vertical columns, exactly as the arithmetic had predicted.

The demonstration is complete: no display, no character reading, no Bad Line. The processor recovers all of its 63 cycles per line. This is also, incidentally, the trick that every C64 programmer uses when they have a big calculation to do: switch the screen off, calculate, switch it back on. The gain is modest and it can be computed — twenty-five Bad Lines per frame, forty cycles each, a thousand cycles recovered out of the 19,656 in a frame: **about 5%**, or 8% if you only look at the display area. It's not much, but it's free.

Patience, and a prediction verified to the pixel

One question of method remains: how do you wait for a *chosen* number of cycles? Our watch loops wait for a line; soon we will need to wait “twelve cycles,” no more.

New instruction — DEX (“DEcrement X”): subtracts 1 from the X register. It costs 2 cycles, and — like cmp — notes whether the result is zero: that is what bne checks right after.

Paired with bne, it gives the most economical countdown on the machine:

```

        ldx #2                ; 2 cycles: the length of the patience
attente dex                ; 2 cycles on each pass
        bne attente          ; 3 if we loop back, 2 the last time

```

Let's add this patience to our stopwatch — screen off, to measure without Bad Lines:

```

!to "patience.prg", cbm

* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
        sei                ; silence
        lda #$0b           ; screen off: no Bad Line
        sta $d011          ; will come to disturb the measurement

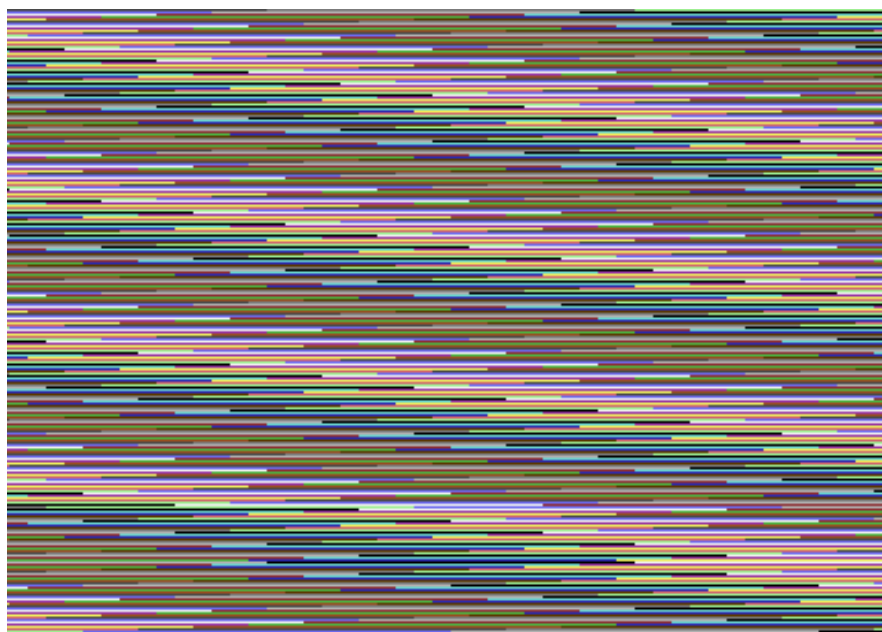
boucle  inc $d020           ; 6 cycles
        ldx #2            ; 2 cycles: the length of the patience
attente dex                ; 2 cycles on each pass
        bne attente        ; 3 if we loop back, 2 the last time
        jmp boucle         ; 3 cycles
                        ; total: 6+2+(2+3)+(2+2)+3 = 20 cycles

```

The cost per stripe becomes: 6 (inc) + 2 (ldx) + 5 (first pass, branch taken), then 4 (last pass) and 3 (jmp) = **20 cycles**.

And 20 does not divide 63. That is the whole point: $63 = 3 \times 20 + 3$. Each line therefore ends 3 cycles “ahead” of the pattern, and the stripes of the next line will be shifted by 3 cycles — that is, at eight pixels per cycle, by **24 pixels**. Our vertical columns should turn into a staircase with a slope of exactly 24 pixels per line.

What you observe:



Real capture. The prediction was: 24 pixels of offset per line. Measured in the image: 24 pixels, to the left, over the whole height.

I took the measurement rather than ask you to believe me: the median offset between two consecutive lines is **exactly —24 pixels**, and the number of stripes per line dropped to 2 or 3 (20 cycles make 160 pixels, so two and a half fit in the visible width). The arithmetic of chapter 3 predicted the image to the pixel.

This is the moment to take stock of what we have just acquired. We can count cycles, we can spend a chosen number of them, and we know that the VIC will steal 40 from us every eight lines. We are no longer spectators of time: we hold it.

Under the hood — the Bad Line has an exact definition, with three simultaneous conditions (line window, equality with the vertical scroll value, display on at the right moment), and the processor freeze goes through two distinct signals (BA then AEC) whose three-cycle offset will have a spectac-

ular consequence in chapter 9. All of this is specified in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 1, §5 (“Bad Lines — exact definition”); the BA and AEC bus signals are in §1 of the same chapter, and the cycle-by-cycle count of the theft in chapter 2.

Next chapter

The VIC has just stolen forty cycles to go and read “the next line of text.” We have talked about these characters without ever looking at them. Where do they come from? Where are they stored? And what if we wrote into them ourselves?

Chapter 5 — The Cells: Where Characters Come From

The question

Before you have even typed anything, there is text on the screen: the welcome message, `READY.`, the blinking cursor. Somebody must have written it. And in chapter 4, we saw the VIC freeze the processor for forty-odd cycles, every eight lines, to fetch forty bytes. Forty bytes **of what**, exactly, and **where**?

The answer is almost disappointingly simple: the characters on the screen are **a thousand bytes of ordinary memory**, in the storeroom, like the others. Write a number into one of them, and a character appears — no system call, no `PRINT`: just an `sta`. And at the end of the chapter, we will move the whole screen elsewhere in memory by writing **a single byte**.

The experiment — a thousand lockers

The first locker of the screen is number 1024, which we will write `$0400`. There are a thousand of them in a row, up to 2023 (`$07e7`). Here is how to fill them all.

```

;-----
; 20 MILLISECONDS – chapter 5: the video matrix, by hand
;
; A thousand lockers starting at $0400: one byte dropped = one
; character displayed. No PRINT, no call to the system.
; We paint the whole screen, then write three letters by hand
; in the top-left corner.
;
; Assemble :   acme matrice.a
; Run :       LOAD"MATRICE",8,1 then RUN
;             (RUN/STOP + RESTORE to get back to BASIC)
;-----
!to "matrice.prg", cbm

* = $0801          ; the stub from chapter 1: 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
sei                ; the BASIC cursor won't come blinking over it

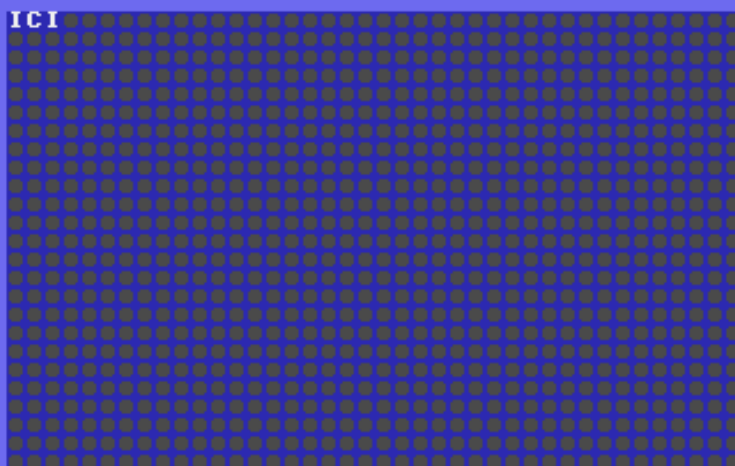
; --- the 1000 cells, in four passes of 256 ---
ldx #0
rempli lda #81      ; SCREEN code 81: the solid disc
      sta $0400,x    ; cells 0 to 255
      sta $0500,x    ; cells 256 to 511
      sta $0600,x    ; cells 512 to 767
      sta $06e8,x    ; cells 744 to 999 (24 overlap)
      lda #11       ; 11: dark gray
      sta $d800,x    ; the color of cell 0, then 1, then 2...
      sta $d900,x
      sta $da00,x
      sta $dae8,x
      dex
      bne rempli

; --- three bytes by hand: the top-left corner is $0400 ---
      lda #9        ; I
      sta $0400
      lda #3        ; C
      sta $0401
      lda #9        ; I
      sta $0402
      lda #1        ; white
      sta $d800     ; ...for these three cells only
      sta $d801
      sta $d802

fini   jmp fini      ; we stop here: the picture stays as it is

```

What you observe:



Real capture (C64 Ultimate, native video output).

The welcome message has disappeared, covered over: the entire screen — twenty-five rows, forty columns — is tiled with the same solid disc, dark gray on a blue background. And in the top-left corner, three white letters, **ICI**, which designate the locker \$0400: the first byte of the matrix is displayed there, the next one \$0401 right beside it, the fortieth ends the first row, the forty-first starts the second. A thousand lockers read in sequence, the way you read a page — and not one new instruction for that: this chapter brings a **map**.

The **video matrix** is an array of **1000 bytes** in ordinary memory, starting at \$0400 (1024) by default. Byte number n decides which character is displayed in cell number n , starting from the top-left corner, left to right then top to bottom. $25 \text{ rows} \times 40 \text{ columns} = 1000$.

Nothing else: no display command, no protocol. The VIC reads these thousand lockers and draws what it finds there, fifty times a second, whether you have filled them or not.

The old hand's elegance: 1000 lockers in four passes of 256

Our loop deserves a word, because it is an absolute classic and its little oddity is deliberate.

Note in passing `sta $0400,x`: the indexed mode from chapter 3 also works **for writing**. “Slot number X of the table” serves to store as well as to load — which is what makes a fill loop so short.

An index register holds a number from 0 to 255: one pass of the loop can therefore cover only 256 lockers. A thousand lockers makes four passes — but $4 \times 256 = 1024$, which is **24 lockers too many**. If the fourth block started at \$0700, it would overflow by 24 lockers *after* the end of the screen (\$07e8–\$07ff), which is precisely where the VIC will go looking for something else in the next chapter. So we start the fourth block 24 lockers **earlier**, at \$06e8: its 256 writes cover cells 744 to 999, the last one landing exactly on the last cell of the screen. The 24 surplus writes fall back on cells that are already painted, with exactly the same value: nothing is damaged, nothing sticks out.

Let's check, because this book never asks you to take anything on faith: $\$06e8 = 1768$, and $1768 - 1024 = 744$; $744 + 256 = 1000$. The count is right.

Two details of the loop are worth stopping for:

- **Why two `lda` per pass?** Because the A register holds only one value at a time. It needs the character code for the four `stas` of the matrix, then the color for the next four. A is a single bucket: we fill it twice per pass.
- **Why `ldx #0` and not `ldx #255`?** Because dex takes X from 0 to 255 (it “wraps around from below”): the first pass handles cell 0, then 255, 254... down to 1, where `bne` stops branching. All 256 values get their turn, in a strange order of no importance whatsoever.

Screen codes are NOT ASCII codes

This is where everybody trips. In the matrix, the letter **A** is not written 65, it is written 1; B is 2, C is 3... Z is 26; space is 32, the digits 0 to 9 are 48 to 57; code 0 gives @ and codes 64 to 127 graphic symbols, which any code + 128 renders in reverse video. Our word “ICI” is therefore written 9, 3, 9. 65 exists too, but it is the BASIC code, the one for `CHR$(65)` and `ASC("A")`: dropped into the matrix it **does not display an A**, since codes 65 to 90 there are graphics. Two tables, two worlds.

Why this double life? Because the number dropped in is not a letter, it is a **drawer number**: the shapes of the characters live in the **character generator**, a piece of furniture with 256 drawers of 8 bytes (one byte per pixel row, 8 rows per cell), where the VIC goes to read *generator base + 8 × code + row number within the cell*.

Color RAM: a thousand more lockers, and a trap

Each cell has **its** color, and that color lives in a separate memory: a thousand other lockers starting at \$d800 (55296), cell n taking the color stored in $\$d800 + n$. Same numbering, constant offset — which is why our loop paints both arrays in the same pass, with the same X. Two peculiarities: only the **four low-order bits** of each locker are used (a color from 0 to 15; this memory is only 4 bits wide), and it **cannot move** — \$d800–\$dbe7, for good. Remember that: in a few pages we will move the screen, and it will stay right where it is.

And the trap, which has cost generations of beginners their time: **a character dropped in without its color can be perfectly invisible.** The screen-clearing routine copies the current **background color** into the thousand color lockers; a character written afterwards into the matrix, without touching Color RAM, is drawn in blue on blue — it is there, the VIC displays it faithfully, and you see nothing. The only way to know what color your character will come out is **to write it yourself.**

These thousand bytes are the Bad Line’s loot

The *shapes*, the VIC reads on every scan line: 40 reads from the character generator, one per column (the “g” accesses). But the *codes* do not change from one pixel line to the next, so it reads them only **once per text row** — and those 40 reads (the “c” accesses) do not have room to fit in its half-microsecond. That is what the Bad Line of chapter 4 is: the processor frozen for forty-odd cycles, and the VIC carrying off **the 40 bytes of the current row.** The theft is double, by the way: each “c” access brings back **12 bits** at once, 8 bits from the matrix and 4 bits from Color RAM, over a bus widened specially for that. And it repeats only one line in eight: between two Bad Lines, the VIC works from an **internal copy** of forty codes and forty colors, so that a character dropped into the matrix does not exist for it until the next Bad Line of its row. To the eye, it is instantaneous; but hold on to the sentence anyway, the last chapters will turn it into an instrument.

Second experiment — the screen is just an address

A naive question, and yet: *why* \$0400? What, in this machine, knows that the screen is there? One register, just one: \$d018. The VIC reads two answers in it.

Bits of \$d018	What they designate	Step
the 4 high bits	where the VIC reads the video matrix	1 KB (1024 bytes)
bits 3, 2 and 1	where it reads the character generator	2 KB
bit 0	nothing, it is unused	—

The four high bits only take sixteen values — so the matrix can only be placed at a multiple of 1024 — and since they occupy the **top** of the byte, the matrix number has to be multiplied by 16: matrix no. 1 (\$0400, the startup one) contributes $1 \times 16 = 16$ to the register, no. 8 (\$2000, the one we are going to build) $8 \times 16 = 128$. The three middle bits are counted the same way, and the startup value is 4: the ROM character generator, which the VIC sees at \$1000. We won’t touch it — we want real letters, not our own shapes (that will be another book, or your next weekend).

Where to put the second matrix? (a decision, not a recipe)

We need a thousand free bytes on a multiple of 1024. Three candidates present themselves, and two disqualify themselves:

- **\$0800** — the immediate neighbor, tempting. **No:** that is where BASIC stores program text (\$0800–\$9fff), and our program *is* a BASIC program, or at least it has the stub of one: it lives at \$0810. Putting the matrix there would mean writing a thousand bytes over the very code that is writing them. Suicide in mid-loop.
- **\$1000–\$1fff** — apparently free, and booby-trapped: in the memory bank where the VIC works by default, that is where it sees the image of the character ROM. It would read shapes there while thinking it was reading codes.
- **\$2000** — 8192, a multiple of 1024, plain RAM, far from our code and far from the character ROM. That is our address. (It is, incidentally, the traditional choice for this kind of need.)

That is the kind of trade-off the machine imposes all the time, and that no manual will make for you: three constraints, one address that satisfies them.

One letter per cell, at chosen places

Our second matrix will not settle for a uniform tiling: we will place seven letters on a diagonal, each at a cell number taken from a table. One more register, then.

New register and new instruction — LDY (“LoaD Y”): loads a byte into the Y register, the third and last working register of the 6510, after A and X. Y serves as an index exactly like X: `sta $2000,y = “drop A into the locker located Y slots after $2000.”` Why a second one? Because a register only does one thing at a time. In the loop that follows, **X counts** (it is the one dex decrements, the one that decides when we stop); it cannot *at the same time* designate the destination cell, which jumps by 41 each time. Two roles, two registers: X counts, Y designates.

Note the form `ldy places,x`: the indexed mode from chapter 3, applied to Y. It is the program that decides the position **while** it runs.

```
;-----
; 20 MILLISECONDS – chapter 5: the screen moves house
;
; We build a SECOND video matrix at $2000, invisible.
; Then a single write to $d018 tells the VIC to go and read
; over there: the screen changes at once, without a single displayed
; character having been rewritten.
;
; Assemble :   acme demenage.a
; Run :       LOAD"DEMENAGE",8,1 then RUN
;             (RUN/STOP + RESTORE to get back to BASIC)
;-----
!to "demenage.prg", cbm

* = $0801                ; the stub from chapter 1: 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                  ; nobody comes to disturb us

; --- 1. the second matrix, at $2000: nobody is looking at it yet ---
    ldx #0
mur   lda #81            ; the same solid disc as in the previous listing
      sta $2000,x        ; cells 0 to 255
      sta $2100,x        ; cells 256 to 511
      sta $2200,x        ; cells 512 to 767
      sta $22e8,x        ; cells 744 to 999
      lda #11           ; dark gray
      sta $d800,x        ; the color, for its part, stays where it has always been
      sta $d900,x
      sta $da00,x
      sta $dae8,x
      dex
      bne mur

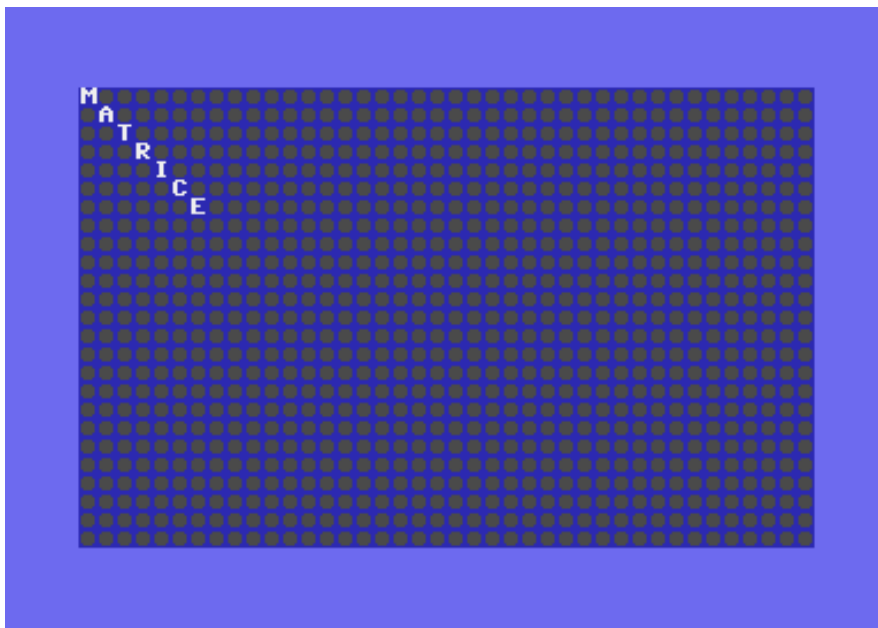
; --- 2. seven letters, each in its place: X counts, Y designates ---
    ldx #7
mot   ldy places,x        ; Y <- the number of the cell where the letter goes
      lda lettres,x       ; A <- its screen code
      sta $2000,y         ; the letter, in matrix no. 2...
      lda #1              ; white
      sta $d800,y         ; ...its color, in Color RAM (which hasn't moved)
      dex
      bne mot

; --- 3. a single write, and the VIC reads elsewhere ---
      lda #132            ; 128 = matrix at $2000 ; 4 = characters at $1000
      sta $d018

fini   jmp fini           ; we stop here: the picture stays as it is

; Slot 0 of both tables is never read: the loop stops at X=1.
places !byte 0,  0, 41, 82, 123, 164, 205, 246  ; a diagonal, one cell per line
lettres !byte 0, 13,  1, 20, 18,  9,  3,  5  ; M A T R I C E
```

What you observe:



Real capture (C64 Ultimate, native video output).

The same wall of gray discs — but it no longer comes from the same place. And on a diagonal from the top-left corner, one cell lower and one to the right each time, the seven white letters: **M A T R I C E**. Throughout the filling, the screen was still displaying the BASIC welcome message: we were writing to \$2000, and nobody was looking at \$2000. Then two instructions, `lda #132` and `sta $d018`, and the picture changed entirely. **Not a single displayed character code was rewritten**: we did not move a thousand bytes, we moved **the VIC's gaze**.

The explanation — $132 = 128 + 4$

The number dropped into `$d018` assembles two answers: **128** in the four high bits (“the matrix is at \$2000”) and **4** in bits 3-2-1 (“the character generator stays the ROM one, at \$1000”). This register carries two unrelated pieces of information, and we only want to move one of them: that is the whole precaution to take with it. A tasty detail in passing: the Commodore documentation recommends `POKE 53272,21` to return to normal where the operating system writes **20**: both are right, because bit 0 is not wired and always reads as 1. The value you write and the one you read back are not necessarily the same.

Now look at the two blocks of `sta` in the first loop: the codes go to \$2000, the colors to `$d800`. Nothing follows them, and the thousand colors are therefore **shared** by every matrix you will ever install — cell number 246 will take the color of the locker `$d800 + 246`, whether its codes come from \$0400 or from \$2000. That is why the welcome message changed color during our filling, and it is what explains the strangest line in the listing:

```

sta $2000,y      ; the letter, in matrix no. 2...
lda #1           ; white
sta $d800,y      ; ...its color, in Color RAM (which hasn't moved)
```

Two `stas`, the same `Y`, two memory regions that have nothing to do with each other — and a single cell on the screen. You have just written, in assembly, the display routine of the Commodore 64.

One honest admission to finish. When our `sta $d018` executes, the beam is in the middle of the frame, and the rows above it have already had their Bad Line: their forty codes come from the **old** matrix. The switch therefore moves down the screen **one text row at a time**, at the pace of the Bad Lines, and it is finished by the end of one frame: twenty milliseconds. That is why the eye only sees one clean change — and that is why a program that wrote `$d018` several times *in the same frame* would display **two different matrices on the same screen**, one above, the other below. Pulling that off requires knowing *where* the beam is to the cycle: that is the program of chapters 7 and 9.

Under the hood — this whole chapter is the gentle version of four sections of *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 1: the video matrix and the two kinds of VIC access (§6.2), the counters that walk through it and the rule of 40 reads during cycles 15 to 54 (§8), the exact address

formulas for the c and g accesses (§9), and \$d018 bit by bit (§2). The screen codes and the prohibition on Color RAM moving come, for their part, from the *Commodore 64 Programmer's Reference Guide* (appendix B, and the chapter “Programming Graphics”).

Next chapter

There are eight bytes we have said nothing about: the ones that follow the end of the matrix — \$07f8–\$07ff when the screen is in its usual place, \$23f8–\$23ff since our move — and which we avoided trampling with our fourth block of 256 writes. They designate shapes that the VIC goes and fetches all on its own, eight times per scan line, and which it displays **on top of** the grid of cells, anywhere, to the pixel. In chapter 6: sprites.

Chapter 6 — Sprites: Eight Free Objects

The question

So far in this book, everything you draw is a prisoner of a grid: a character can only be placed *inside* a cell, and to move a figure by one pixel you would have to redraw the characters it occupies, then erase the background behind it. No game works like that. So how does the Commodore 64 move a spaceship, a ball, a monster around to the exact pixel, over a background it doesn't damage?

The VIC-II's answer is surprisingly generous: next to its character machine, it carries **eight small independent artists**. MOS called them *MOBs* — *Movable Object Blocks*; everybody says **sprites**. Each one carries a drawing **24 pixels wide by 21 tall**, can be placed wherever you want **to the exact pixel**, has its own color, passes over the image without touching it, and doesn't consume a single byte of background. Free? No: they are paid for in **cycles**, and that is the third of this chapter's three experiments — bring a creature to life, line up eight of them, watch them steal time.

This is the longest chapter in the book, and each of its three experiments stands on its own: if you can only run one today, take the first.

The drawing: twenty-one lines of 0s and 1s

Twenty-four pixels wide, one byte of eight bits: **three bytes per sprite line**, and twenty-one lines of three bytes make **63 bytes** — the exact size of a sprite drawing, always. The three bytes are read from left to right, and in each byte the most significant bit is the left pixel. A bit set to **1** = one pixel of the sprite's color; a bit set to **0** = **nothing at all**, transparent, you see the screen through it — which is what gives sprites their silhouette. In hexadecimal, these 63 bytes would be unreadable; fortunately, the assembler knows how to read numbers bit by bit.

New notation — binary %: %01111110 denotes a byte written **bit by bit**, eight digits, one per bit. It is the same number as 126 or \$7e — the same value, written differently. The leftmost digit is **bit 7** (the most significant), the rightmost one is **bit 0**. For a sprite drawing, this is exactly the order of the pixels on screen: left to right. You no longer code, you draw.

And that gives a drawing you can read like a drawing, right in the program's source:

```
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
```

These eight lines are the top of a little round creature: the first five draw a widening skull, then two holes of 0s carved into the 1s, one on each side — the eyes: three 0s wide on the left, three or four on the right depending on the row, since the drawing is not quite symmetrical. The body and a wide open mouth follow the same logic, you will read them in the listing. Take some graph paper, 24 squares by 21, blacken whatever you like, copy it line by line: that is the whole craft, including that of the games you loved.

The experiment — one sprite, motionless, in the center

The VIC asks only four questions about a sprite — **where is its drawing, where on the screen, what color, is it switched on** — and there is a lever-locker for each one.

Where is the drawing? Locker \$07f8. It doesn't contain an address but a **64-byte block number**: we store our drawing at address 2560 (\$0a00), which equals 40 × 64, and we write 40 into \$07f8. The reason for this division comes right after the experiment.

Where on the screen? \$d000 for sprite 0's X, \$d001 for its Y. The screen is 320 pixels wide and a byte only counts up to 255: sprite X therefore has a **ninth bit**, stored separately in \$d010, one per sprite. We want

$X = 172$, which places the drawing's 24 pixels right in the middle of the display window; the value fits in one byte, so sprite 0's ninth bit must be **off**. As for Y , it holds a surprise worth remembering: **the VIC compares the Y coordinate at the end of the previous line**, so the drawing only starts on the *next* line. Write 140, and the first line drawn is 141.

What color? $\$d027$ for sprite 0, a single color for the whole drawing (the VIC can do better, we won't need it here). **Is it switched on?** $\$d015$: one locker, eight switches, bit 0 for sprite 0, bit 7 for sprite 7 — and this is where we are missing two instructions, because we want to switch on **one** switch without touching the other seven.

New instruction — ORA (“OR with Accumulator”): compares register A and a byte bit by bit, and keeps a 1 everywhere **at least one of the two** had a 1. `ora #00000001` **switches on** bit 0 of A and leaves the other seven exactly as they were. The full pattern — read the locker, switch on a bit, write it back — is the everyday gesture of the C64 programmer: `lda $d015 / ora #00000001 / sta $d015`.

New instruction — AND (“AND with Accumulator”): compares register A and a byte bit by bit, and keeps a 1 only where **both** had a 1. In other words, every bit facing a 0 is **switched off**, every bit facing a 1 is **preserved**. `and #11111110` switches off bit 0 and touches nothing else. ORA switches on, AND switches off: with these two, you are master of one switch out of eight.

```

;-----
; 20 MILLISECONDS – chapter 6: a free object
;
; A creature of 24 x 21 pixels, motionless in the center of the screen.
; It floats ABOVE the text: BASIC takes over again, you
; can type, the screen scrolls – it doesn't move by a single pixel.
;
; Assemble :    acme unsprite.a
; Run :        LOAD"UNSPRITE",8,1 then RUN
;-----
!to "unsprite.prg", cbm

* = $0801                ; the chapter 1 stub: 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
; --- where is the drawing? block no. 40, since 40 x 64 = 2560 = $0a00
lda #40
sta $07f8                ; sprite 0's pointer locker

; --- where is the sprite?
lda #172                 ; X = 172: the middle of the window
sta $d000
lda $d010                ; the eight "this X exceeds 255" bits
and #%11111110          ; we switch OFF sprite 0's, the other seven untouched
sta $d010
lda #140                 ; Y = 140: the creature starts at line 141
sta $d001

; --- what color?
lda #7                   ; yellow
sta $d027

; --- and we switch it on
lda $d015                ; the eight enable switches
ora #%00000001          ; we switch ON sprite 0's, the other seven untouched
sta $d015

rts                      ; BASIC takes over again – the creature stays

;-----
; The drawing: 21 lines of 3 bytes = 63 bytes.
; Address MUST be a multiple of 64 ($0a00 = 40 x 64).
; A "1" = one pixel of the sprite's color, a "0" = nothing
; at all: you see the screen through it.
;-----
* = $0a00
dessin
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111110,%11111111,%01111100
!byte %00011111,%00000000,%11111000
!byte %00011111,%11111111,%11111000
!byte %00001111,%11111111,%11110000
!byte %00000111,%11111111,%11000000
!byte %00000011,%11111111,%10000000
!byte %00000001,%11111111,%10000000
!byte %00000000,%01111110,%00000000

```

What you observe:



Fifteen lines, and a yellow creature stands in the middle of the screen. The `rt`s handed control back to BASIC: `READY.` is back, the cursor blinks, the machine is yours — and the creature is still there. Try the test that matters: **type**. The text passes **behind** it, slides under it, disappears at the top; it doesn't move, doesn't erase, doesn't tear. It is not *in* the image: it is *in front of* it.

The explanation — why 64?

Why does the pointer count in blocks of 64 bytes instead of giving an address? Because it is **only one byte**, from 0 to 255, where an address would take two: the VIC doesn't want to know *where*, it wants to know *which one*. And the count comes out round, because it has only fourteen address wires and so sees only **16 kilobytes** at a time — a “bank”, the one that starts at address 0 as long as you don't touch it. Now **256 blocks × 64 bytes = 16,384 bytes**, exactly those 16 kilobytes. Remember: the pointer is **the drawing's address divided by 64**, and the address must therefore be a **multiple of 64**. $2560 \div 64 = 40$. (A drawing is 63 bytes and a block 64: the last byte is lost, that is the price of round numbers.)

And why `$07f8`? Because the eight pointers are stored **right behind the screen**: chapter 5's video matrix occupies a thousand lockers starting at `$0400`, and the eight last bytes of the kilobyte that contains it — `$07f8` to `$07ff` — are the pointers of sprites 0 to 7. They follow the screen like its shadow: move the matrix, the pointers follow it.

That leaves the display mechanism, which explains everything else in the chapter. On **each raster line**, the VIC reads the sprite's pointer then, if the sprite is visible on that line, its **three bytes** for the current line; it loads them into a **24-bit shift register** and, as soon as the beam reaches the sprite's X coordinate, it pushes this register to the left, **one bit per pixel** — the first bit out is bit 7 of the first byte, the left pixel. Two consequences govern every Commodore 64 game: after 24 pixels the register is **empty**, so a sprite cannot be used twice on the same line (**eight sprites per line, not nine**); but lower in the image, a sprite that has finished its display **can be reused**, you only need to change its Y. This is how games display twenty objects with eight sprites, at the price of tight timing.

The experiment — all eight, in a row

Since there are eight, let's line them up. The geography of the lockers is mechanically regular: positions go in pairs from `$d000` to `$d00f` (**even = X, odd = Y**), colors from `$d027` to `$d02e`, pointers from `$07f8` to `$07ff`, one switch each in `$d015`, one ninth bit of X each in `$d010`. The listing that follows writes its thirty-four values one by one, with no loop and no indexed table: it is long to read, but nothing is hidden in it. Just one subtlety: the centered row puts the eighth sprite at X = 284, beyond 255 — so its ninth bit comes into play, and only that one.

```

;-----
; 20 MILLISECONDS – chapter 6: all eight, in a row
;
; The eight sprites, same drawing, eight colors, lined up in the middle
; of the screen. No tricks: each position is written by hand,
; locker by locker. It is long to read, but nothing is hidden.
;
; Assemble :    acme huit.a
; Run :        LOAD"HUIT",8,1 then RUN
;-----
!to "huit.prg", cbm

* = $0801                ; the chapter 1 stub: 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    lda #dessin / 64      ; the assembler does the division: 2560 / 64 = 40
    sta $07f8             ; the eight pointers, one per sprite...
    sta $07f9
    sta $07fa
    sta $07fb
    sta $07fc
    sta $07fd
    sta $07fe
    sta $07ff             ; ...all on the SAME drawing

    lda #140              ; same Y for all eight: the same row
    sta $d001
    sta $d003
    sta $d005
    sta $d007
    sta $d009
    sta $d00b
    sta $d00d
    sta $d00f

    lda #60               ; the X values, one by one, 32 pixels apart
    sta $d000
    lda #92
    sta $d002
    lda #124
    sta $d004
    lda #156
    sta $d006
    lda #188
    sta $d008
    lda #220
    sta $d00a
    lda #252
    sta $d00c
    lda #28               ; the eighth: 284 = 256 + 28
    sta $d00e
    lda #%10000000        ; ...its 9th bit, and that one only
    sta $d010

```

(All eight, in a row: here is the **rest of the same file**, to paste directly after the first part. The two halves make up a single program.)

```

        lda #1                ; eight colors, one per sprite
        sta $d027             ; white
        lda #7
        sta $d028             ; yellow
        lda #8
        sta $d029             ; orange
        lda #10
        sta $d02a             ; light red
        lda #13
        sta $d02b             ; light green
        lda #3
        sta $d02c             ; cyan
        lda #14
        sta $d02d             ; light blue
        lda #4
        sta $d02e             ; pink

        lda #%11111111       ; all eight switches at once
        sta $d015
        rts

; -----
; The drawing: 21 lines of 3 bytes = 63 bytes.
; Address MUST be a multiple of 64 ($0a00 = 40 x 64).
; A "1" = one pixel of the sprite's color, a "0" = nothing
; at all: you see the screen through it.
; -----
* = $0a00
dessin
        !byte %00000000,%01111110,%00000000
        !byte %00000001,%11111111,%10000000
        !byte %00000011,%11111111,%11000000
        !byte %00000111,%11111111,%11100000
        !byte %00001111,%11111111,%11110000
        !byte %00011100,%01111110,%00011100
        !byte %00011100,%01111110,%00011100
        !byte %00111100,%01111110,%00111100
        !byte %00111111,%11111111,%11111100
        !byte %00111111,%11111111,%11111100
        !byte %00111111,%11111111,%11111100
        !byte %00111111,%11111111,%11111100
        !byte %00111111,%11111111,%11111100
        !byte %00111110,%11111111,%01111100
        !byte %00011111,%00000000,%11111000
        !byte %00011111,%11111111,%11111000
        !byte %00001111,%11111111,%11110000
        !byte %00000111,%11111111,%11100000
        !byte %00000011,%11111111,%10000000
        !byte %00000001,%11111111,%10000000
        !byte %00000000,%01111110,%00000000

```

What you observe:



Eight creatures, eight colors, a perfectly straight row across the screen. The pointer is the same for all eight: **a single drawing in memory, eight copies on screen**, each sprite carrying only its coordinates, its color and its switch. Notice also what has disappeared: no more ora, no more and. When you write all eight bits at once — `lda #%11111111` then `sta $d015` — no mask is needed; ORA and AND are for when you want to change **only one** while respecting the others, which is almost always the case in a real program.

Two rules of cohabitation before we make them move. When they overlap, **the lowest-numbered sprite goes in front**: the hierarchy is hard-wired, sprite 0 always wins, sprite 7 always loses. And **the border beats everybody** — a sprite that enters it disappears underneath, unless you open it, which is a whole chapter in itself (chapter 8, where we will open the top and bottom borders; the side borders demand cycle-exact work on every line and fall outside the scope of this book).

The experiment — the eight thieves

Now, the bill. Let's take the visual stopwatch from chapter 4 again: a loop that does nothing but increment a color, so that **each write leaves a color boundary** at the precise spot where the beam was — where the processor advances normally, these boundaries form perfectly straight columns; where cycles are stolen from it, they shift. Two adjustments this time. We paint **the background too** (\$d021), so that the stripes cross the sprite area. And we time the loop to **21 cycles**: `inc` on an absolute locker costs 6 cycles, `jmp` costs 3, so $6 + 6 + 6 + 3 = 21$. Now $21 \times 3 = 63$, the exact length of a PAL raster line: the pattern repeats identically on every line, and anything that doesn't get its full 63 cycles will shift it. That is our theft detector.

One last adjustment, a cosmetic one: our eight creatures turn **black**. This is not a malfunction — it is the only way to still tell them apart. The background now changes color on every loop turn — about every twenty-one microseconds — and runs through all sixteen shades of the palette; any sprite color would drown in it at times. Black, on the other hand, stands out against the other fifteen. So you will see eight silhouettes, and that is intended.

(This listing begins with the setup of the eight sprites, identical to the previous one; what is new lies in the last four lines, the stopwatch ones.)

```

;-----
; 20 MILLISECONDS – chapter 6: the eight thieves
;
; The visual stopwatch from chapter 4 (inc in a loop), widened to
; the screen background and timed to 21 cycles – with the eight sprites
; laid across the middle of the image. Watch the color columns
; at the height of the creatures.
;
; Assemble :   acme voleurs.a
; Run :       LOAD"VOLEURS",8,1 then RUN
;             (RUN/STOP + RESTORE to exit)
;-----
!to "voleurs.prg", cbm

* = $0801                ; the chapter 1 stub: 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                    ; silence: nobody interrupts us any more

    lda #dessin / 64       ; the eight pointers on the same drawing
    sta $07f8
    sta $07f9
    sta $07fa
    sta $07fb
    sta $07fc
    sta $07fd
    sta $07fe
    sta $07ff

    lda #140               ; same Y: all eight on the same row
    sta $d001
    sta $d003
    sta $d005
    sta $d007
    sta $d009
    sta $d00b
    sta $d00d
    sta $d00f

    lda #60               ; the X values, one by one
    sta $d000
    lda #92
    sta $d002
    lda #124
    sta $d004
    lda #156
    sta $d006
    lda #188
    sta $d008
    lda #220
    sta $d00a
    lda #252
    sta $d00c
    lda #28               ; the eighth: 284 = 256 + 28
    sta $d00e
    lda #%10000000
    sta $d010

```

(The eight thieves: here is the *rest of the same file*, to paste directly after the first part. The two halves make up a single program.)

```

lda #0                ; all eight BLACK: the only color that stands out
sta $d027             ; against a background running through the 16 shades
sta $d028
sta $d029
sta $d02a
sta $d02b
sta $d02c
sta $d02d
sta $d02e

lda #%11111111       ; all eight switched on
sta $d015

; --- the stopwatch: 6 + 6 + 6 + 3 = 21 cycles per turn,
; and 21 x 3 = 63 = the whole line. A nicely vertical pattern.
boucle inc $d020       ; 6 cycles – the border changes color
inc $d021             ; 6 cycles – the background too
inc $d020             ; 6 cycles – the border again
jmp boucle            ; 3 cycles – and start over

;-----
; The drawing: 21 lines of 3 bytes = 63 bytes.
; Address MUST be a multiple of 64 ($0a00 = 40 x 64).
; A "1" = one pixel of the sprite's color, a "0" = nothing
; at all: you see the screen through it.
;-----
* = $0a00
dessin
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111110,%11111111,%01111100
!byte %00011111,%00000000,%11111000
!byte %00011111,%11111111,%11111000
!byte %00001111,%11111111,%11110000
!byte %00000111,%11111111,%11100000
!byte %00000011,%11111111,%11000000
!byte %00000001,%11111111,%10000000
!byte %00000000,%01111110,%00000000

```

What you observe:



The entire screen has become a stopwatch: fine horizontal stripes (the color advances a little on each line) cut by **large vertical boundaries**, each one being an inc executed. Three things to read in it.

At the top, the boundaries are perfectly vertical. In the top border, the VIC is still working — it reads idly, it even fetches the pointers of the eight sprites — but all of that fits within its own half-microsecond: it **requisitions** nothing. The processor keeps its 63 cycles, and the 21-cycle loop lands in the same place on every line, to the pixel.

In the display area, the boundaries climb in small steps. A step of **eight pixels — one cycle — every eight lines**: this is chapter 4's Bad Line, unflappable. Why such a small step, when a Bad Line steals some forty cycles? Because our stopwatch doesn't measure the theft: it measures what **exceeds** a whole number of turns, and a single hand doesn't count the turns. That remainder can be calculated, and it lets us tighten the measurement — provided we are honest about what it proves. A one-cycle step tells us only that the cycles left to the processor land **within one cycle of a multiple of 21**: that could be 20, 22, 41, 43... The corresponding theft would then be 43, 41, 22 or 20 cycles. Let's cross-check with what the documentation says — a Bad Line takes between 40 and 43 cycles — and only **two** remain: 41 or 43.

We won't know which, and that is just fine: the lesson is right there. A measurement alone left four possibilities, a documented range alone left four as well (40, 41, 42, 43), and the two together leave only two. We gained precision without making anyone say more than they know — and to decide between 41 and 43, we would need an instrument finer than a screen capture.

Above all, remember what this range means: **the theft is not a fixed number**. It depends on where the processor was in its instruction when the VIC raised its hand. The same Bad Line doesn't cost quite the same thing from one line to the next — and in chapter 9, this small irregularity will decide the fate of three programs.

And on the band of the creatures, the whole pattern is pushed sideways by one block. No more small steps: a sharp shift, on the order of a hundred pixels, over about twenty lines — the twenty-one of the drawing, plus one just above. The sprites, motionless, silent, without a single instruction of their own, have just stolen time from the processor.

The explanation — what a sprite really costs

A sprite makes two kinds of memory access. **The pointer** first: a read of its locker (\$07f8 for sprite 0, \$07f9 for sprite 1, and so on) on **every** raster line, in the **first** half of the cycle — the VIC's half. So it costs the processor nothing, and it happens **even if the sprite is off**: a sprite that is off is truly free. **The three bytes of the drawing** next, only if the sprite is displayed on that line: they occupy the **three half-cycles that immediately follow** the pointer read, that is, the second half of the current cycle then both halves of the next cycle. The first and the third of these reads fall in the **processor's half**. It loses them.

Hence the figure to remember: **about two cycles lost per sprite displayed, per raster line** — eight sprites on the same line, sixteen cycles. Plus a supplement, because the VIC is a polite craftsman: when it needs the 6510's half, it **warns three cycles ahead** (the BA signal), and during those three cycles the processor can still finish writes but not *start* a new instruction. The theoretical count is therefore $16 + 3 = 19$ **cycles stolen**. In practice fewer, because the processor nibbles away part of the warning window: the reference tables give **46 to 49 cycles available** out of the 63 of a line carrying eight sprites, that is **14 to 17 cycles stolen**. Note the range: the primary sources don't agree to the cycle, and they say so. With this kind of number, honesty is an interval, not a value.

And now, check. The beam covers **8 pixels per cycle**: 14 to 17 stolen cycles make 112 to 136 pixels of shift, exactly the order of magnitude of the jump before your eyes. You have just read a cycle theft directly on the screen, with a graduated ruler and nothing else. Here, finally, is the machine's worst case, the one every demo programmer knows by heart: **a Bad Line that also carries all eight sprites leaves only 4 to 7 cycles to the processor**. Out of sixty-three. Enough room for two short instructions, no more: this is the absolute limit of the Commodore 64, and it is against it that ambitious effects break.

Two clarifications on the exact shape of what you see. **Twenty-two lines and not twenty-one**, because the VIC reads ahead of the phase: the data of sprites 3 to 7 are read at cycles 1 to 9 of the line where they are displayed (the appendix, at the end of the book, gives the complete map of the 63 cycles of a line), but those of sprites 0, 1 and 2 at cycles 58, 60 and 62 — **in the previous line**. And if the whole pattern of the line is shifted, not just the piece located behind the creatures, it is because those cycles fall outside the image (the line return on the left, the extreme right edge): the processor takes its delay **before** entering the visible part and keeps it until the end. We don't see the theft, we see its effect.

One last curiosity, counter-intuitive and verified: **switching off a sprite "in the middle" gives back no cycle**. Switch off sprite 4 while all eight are active (`lda $d015 / and #$11101111 / sta $d015`): the pattern

doesn't move by a pixel, because during the slot that sprite 4 would have occupied, the VIC is already announcing that it wants the bus for sprite 5. The windows are contiguous; to win back time, you have to free a **range** of neighboring slots, not a random sprite. Eight free objects, then — but in a Commodore 64, time belongs to the video chip, and sprites are one of the most elegant ways of giving it to the VIC.

Under the hood — the complete life cycle of a sprite (the test of its Y coordinate, the switching on of its DMA, its internal counters, its shift register) and the priority rules between sprites are in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 1, §10. The accounting of the cycle theft — the two lost reads out of three, the three-cycle warning, and the table of budgets per line type (63 / 46-49 / 23 / 4-7 cycles) — takes up its entire chapter 2, after Pasi Ojala (1992) and the measured timing diagrams of Marko Mäkelä (1994). The cost of the instructions in our stopwatch: chapter 3, §2.

Next chapter

You now know how to manufacture delay — twenty-two lines, with eight sprites that do nothing. In chapter 7, we will do the opposite: **manufacture time**. You now have all thirteen instructions of the book; what comes next needs no new tool, only knowing two hidden VIC counters — which nobody can read, but which everybody can fool. By the end of the chapter, the screen will fall.

Chapter 7 — The Hidden Counters, and the Screen That Falls

The question

We are now entering the third part of the book, the one where we stop observing the machine and start hijacking it. And the first question to ask it is this: we know how to make a character appear by writing into a locker. Can we make the **whole** screen fall without moving a single byte?

The answer is yes, it fits in a loop of six lines, and it relies entirely on the Bad Line from chapter 4.

The VIC's bookmark

First we need to understand something we have carefully steered around until now: the VIC does not know “where it is” on the screen the way you and I would imagine it.

It keeps two internal counters, invisible from the program, and it is this pair that does all the work. The first remembers **which line of text** it is currently displaying — let's call it its bookmark. The second remembers **which line of pixels**, from zero to seven, inside that line of text.

Here is the crucial point, and it follows directly from chapter 4: **without a Bad Line, no new row of text is read**. The Bad Line is not just the moment when the VIC steals cycles: it is its *starting signal* for a row. No Bad Line, no read — the VIC stays where it is.

A nuance to keep in reserve, because chapter 9 will make it its business: the Bad Line triggers the **reading** of a row, but the bookmark only moves from one row to the next once the eight pixel lines are used up. Bad Line and bookmark advance are two distinct events; here they go together, and that is all we need.

Now, we know exactly what triggers a Bad Line: equality between the last three bits of the line number and the vertical scroll value, that eight-position setting kept in `$d011`. A register we can write to, line after line, as often as we like.

The idea behind the trick then formulates itself: **if the equality never occurs, the VIC reads nothing new anymore**. It waits, the display stays suspended, and when we finally give it its freedom back, it resumes reading where it left off — forty lines further down. Seen from the screen, the whole content has fallen.

This trick has a name, given by the demo programmers of the 1980s: **FLD**, for *Flexible Line Distance*. It is the simplest of the great C64 tricks, and a good way in before chapter 9.

The offset trap, and why you must aim two lines ahead

Our loop is going to read the current line number and write a forbidden setting. Question: which setting? The natural reflex would be to write a setting different from the current line — say $(\text{line} + 1) \& 7$. That is wrong, and it is an instructive mistake. Think about what happens while line i is in progress: our loop writes the setting $(i+1) \& 7$ there. Then line $i+1$ begins — and the Bad Line test is redone on every cycle; and in the first cycles of the line, when the VIC decides whether to take over the bus, our loop has not had time to run again. The value still in place is $(i+1) \& 7$, and the line number is now $i+1$. Equality. Bad Line. Trick failed.

So you must aim **two lines ahead**: write $(i+2) \& 7$ during line i . This value is equal neither to $i \& 7$ (no Bad Line right away), nor to $(i+1) \& 7$ (no Bad Line at the start of the next line, even if our loop is late). The “+2” is not a superstition: it is exactly the margin we need so that our loop's lateness costs us nothing. Remember the reasoning more than the formula. Programming the VIC means constantly asking yourself: *what is this register worth at the instant the machine looks at it?* — and not at the instant we write it.

The experiment

```
!to "chute.prg", cbm

DEPART = $32          ; line where the fall starts (50)
FIN     = $5a          ; line where we hand back control (90) -> 40 lines

* = $0801             ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence
    lda #0             ; what the VIC will re-read at rest, at $3fff:
    sta $3fff          ; zero, hence no black stripes

; --- wait for the top of the screen ---
trame   lda $d012
        cmp #DEPART
        bne trame

; --- for N lines: move the setting so that equality NEVER happens ---
chute   ldx $d012       ; 4 : the current line
        lda cran,x      ; 4 : a forbidden setting for the TWO lines to come
        sta $d011       ; 4 : no more Bad Lines
        lda $d012       ; 4 : are we at the end?
        cmp #FIN        ; 2
        bne chute      ; 3 -> 21 cycles per pass, 3 passes per line

; --- we hand back control: Bad Lines resume, the screen resumes reading ---
        lda #$1b        ; setting 3, screen on, 25 lines
        sta $d011
        jmp trame       ; and start over at the next frame

; --- the table of "forbidden" settings ---
!align 255, 0
cran
!for i, 0, 255 {
    !byte $18 | ((i + 2) & 7) ; $18 = screen on + 25 lines
}
```

Three assembler notations appear here for the first time, and they cost nothing to understand. `DEPART = $32` gives a **name** to a number: the assembler will replace `DEPART` with `$32` everywhere, and you can change the value in a single place. `!for i, 0, 255 { ... }` is a **manufacturing** loop: it does not run in the machine, it runs in the assembler, and its result is a 256-byte table written into the program. Finally, `&` and `|` are the same operations as the `and` and `ora` instructions from chapter 6, but computed at manufacturing time.

And above all, the key to reading this table: **`(i + 2) & 7` keeps the last three bits of `i + 2`** — exactly those three bits that chapter 4 said trigger the Bad Line when they land on the vertical scroll value. The `| $18` switches back on, on top, the switches we want to keep. That little line *is* the trick.

Recognize the signature pattern from chapter 3 — `ldx $d012` followed by `lda cran,x` — but put to work on something other than color. The whole trick was computed by the assembler, in advance, in the `cran` table: at display time, all that is left is to read and write. The loop costs 21 cycles, so it runs three times per line, and this generosity is our safety: even if one of the three passes lands at the wrong moment, the other two hold the register.

Finally, note the `$18` in the table: it switches the screen back on and keeps the window at 25 lines on every write. A hardware register is rewritten **in full** — you cannot touch a single switch without also saying what you want for the others. We would know how to change just one, with the `and/ora` from chapter 6; here it is pointless, since the table knows in advance the complete value to write.

What you observe:



Real capture. The text is intact, in its place in memory — it is its reading that was suspended for forty lines.

Measured on the capture: the first row of text, which normally starts at line 43, starts here at line 83. **Exactly forty lines** — the difference between our constants FIN and DEPART. Change FIN, and the fall changes by the same amount.

And the emptiness above? It is not black, nor a curtain: it is the VIC which, deprived of a new line of text, enters what the documentation calls its **idle state**. It keeps conscientiously reading one address, always the same — the byte \$3fff — and displays its bits as pixels, with no more color coming from the matrix, which it no longer reads: in text mode, a 1 bit comes out black, a 0 bit comes out as background color. That is why our program starts by storing **zero** in that byte: the band is then uniform, background color, neatly displayed. Had the byte held anything else, black stripes would appear in place of the plain background. The VIC did not “do nothing”: it displayed emptiness with diligence.

What you have just gained

Take the measure of the change. Up to chapter 6, we wrote into lockers to tell the machine *what* to display. Here, for the first time, we wrote into a locker to tell it *when* — and we obtained an effect that its hardware offers nowhere: a vertical scroll of the whole screen, at almost no cost, without moving a byte of memory. That is the very definition of C64 tricks. You do not find a hidden function: you observe a rule (“the bookmark only advances on a Bad Line”), you notice that one of its ingredients is under our control, and you use it at the wrong time.

Under the hood — the exact mechanism (the VC and RC counters, the reset of the line counter, the transition between the display state and the idle state, and the complete catalog of the effects that follow from it: FLD, Linecrunch, VSP) is specified in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 1: §8 for the counters VC and RC, §7 for the idle state, and §16.2 for FLD itself.

Next chapter

We made the screen fall by preventing an event from happening. In the next chapter, we will make the **border** disappear by preventing two comparisons from landing right — and a sprite will wander where the screen does not exist.

Chapter 8 — Opening the Border

The question

There is a frame around the Commodore 64's picture. Some thirty lines at the top, as many at the bottom, a band on each side — a zone where, they say, “you can't display anything”.

And yet the demos of the 1980s make sprites roam there. How?

From this chapter on, I owe you a reassuring warning: **you will not learn any new instruction**. The thirteen you know will be enough until the last page, FLI included. Everything that comes now is not a matter of tooling, but of understanding the hardware — and you already know enough.

The border is not a zone: it is a tap

Here is the idea you have to unlearn. We naturally imagine the border as a region of the screen, something geographic: “here is the window, there is the frame”. The VIC does not work like that at all.

It keeps a **switch** — electronics people call it a flip-flop — that answers a single question: “right now, do I output the border color?” When this switch is at 1, the VIC covers everything: graphics, sprites, absolutely everything. When it is at 0, the image comes through.

And this switch is operated by **comparisons**. Two interest us:

- when the line number reaches the **bottom** comparison — 251 in 25-row text mode — the flip-flop is **set to 1**, and the bottom border begins;
- when it reaches the **top** comparison — 51 in the same mode — the flip-flop is **reset to 0**, and the display window begins.

Now note the sentence from the documentation that contains the whole trick, and read it twice: *the comparison is only true if the value is reached exactly — it is not a range test*.

The VIC does not ask itself “am I below line 251?” It asks itself “am I **at** line 251?” And if the answer is no, nothing happens. Ever. There is no catch-up session.

Two values, one switch

What remains is to find out how to dodge a comparison. We cannot stop the beam from reaching line 251 — but we can change **the value it compares against**.

The C64 indeed offers two window heights: 25 rows of text (the usual setting) or 24 rows, one setting tighter. This choice sits in a single switch of the `$d011` locker, and it changes both compared values:

Mode	Top comparison	Bottom comparison
25 rows	line 51	line 251
24 rows	line 55	line 247

The plan then writes itself. Stay in 25 rows while line 247 goes by: the compared value is 251, the equality is false, nothing happens. Then, before line 251 arrives, switch to 24 rows: the compared value becomes 247... but that line is already behind us. The equality is false a second time.

Two comparisons dodged, and the flip-flop is never set to 1. **The bottom border will not take place**, not because we erased it, but because nobody gave the order to turn it on.

Changing one switch without touching the others

A point of method, promised in chapter 7. The `$d011` locker contains eight switches that have nothing to do with each other: screen on, window height, vertical scroll value... Yet you cannot write just one of them: every write replaces all eight.

This is where the two instructions from chapter 6 come in:

```
lda $d011
and #%11110111      ; this zero TURNS OFF the switch, the other seven untouched
sta $d011
```

Read, modify one bit, write back. and with a zero turns off; ora with a one turns on. This is the most common gesture in all of C64 programming, and you have just seen it in its most natural use: **respecting what your neighbor has set**.

The experiment

```

!to "sansbord.prg", cbm

* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
; --- a creature, placed BELOW the bottom of the screen ---
lda #40                  ; shape at block 40 ($0a00)
sta $07f8
lda #160
sta $d000                ; X
lda $d010
and #%11111110          ; X does not exceed 255
sta $d010
lda #250                 ; Y = 250: in the bottom "border"
sta $d001
lda #7                   ; yellow
sta $d027
lda $d015
ora #%00000001          ; sprite 0 on
sta $d015

lda #0                   ; the ghost byte: see the end of the chapter
sta $3fff

sei                      ; silence

; --- every frame: keep BOTH comparisons from landing right ---
attend1 lda $d012
cmp #250                 ; line 251 is about to arrive...
bne attend1
lda $d011
and #%11110111          ; ...we switch to 24 lines: the compared
sta $d011                ; value becomes 247, already past

attend2 lda $d012
cmp #252                 ; line 251 went by without triggering anything
bne attend2
lda $d011
ora #%00001000          ; back to 25 lines for the next frame
sta $d011

jmp attend1

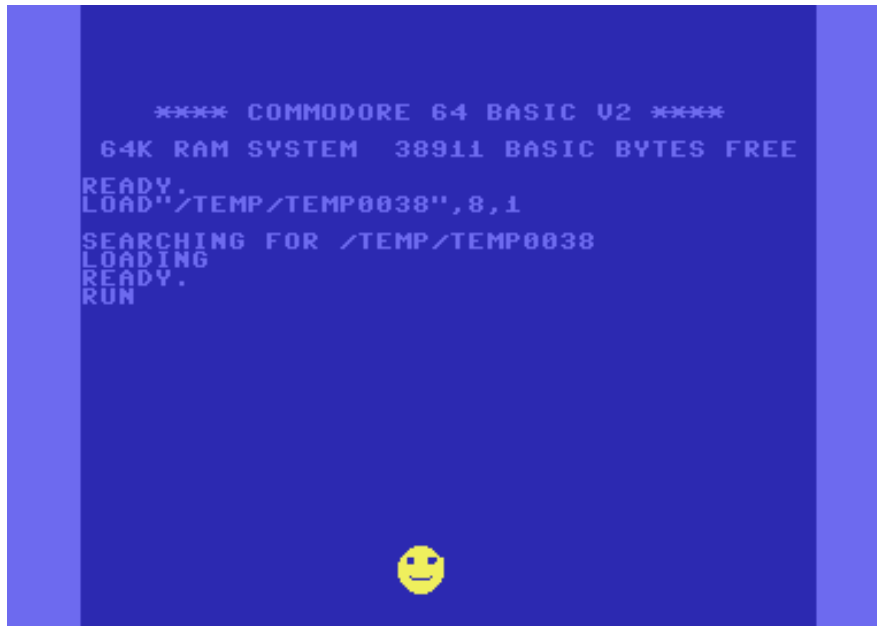
;-----
; The shape: 21 lines of 3 bytes = 63 bytes.
; Address MUST be a multiple of 64 ($0a00 = 40 x 64).
; A "1" = one pixel of the sprite color, a "0" = nothing
; at all: you see the screen through it.
;-----
* = $0a00
dessin
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111110,%11111111,%01111100
!byte %00011111,%00000000,%11111000
!byte %00011111,%11111111,%11111000
!byte %00001111,%11111111,%11110000
!byte %00000111,%11111111,%11100000
!byte %00000111,%11111111,%11100000
!byte %00000011,%11111111,%11000000
!byte %00000001,%11111111,%10000000
!byte %00000000,%01111110,%00000000

```

The sprite shape is the one from chapter 6, reprinted at the end of the listing so that you can type it — or paste it — in one go. Note in passing its `* = $0a00` line: it is not decorative. Without it, the 63 bytes would

be stored right after the code, and the sprite would go looking for its shape where there is nothing. Notice the position: $Y = 250$, that is, *below* the bottom limit of the screen. In a machine that respects its own rules, this creature is invisible.

What you observe:



Real capture. The creature is where the screen does not exist.

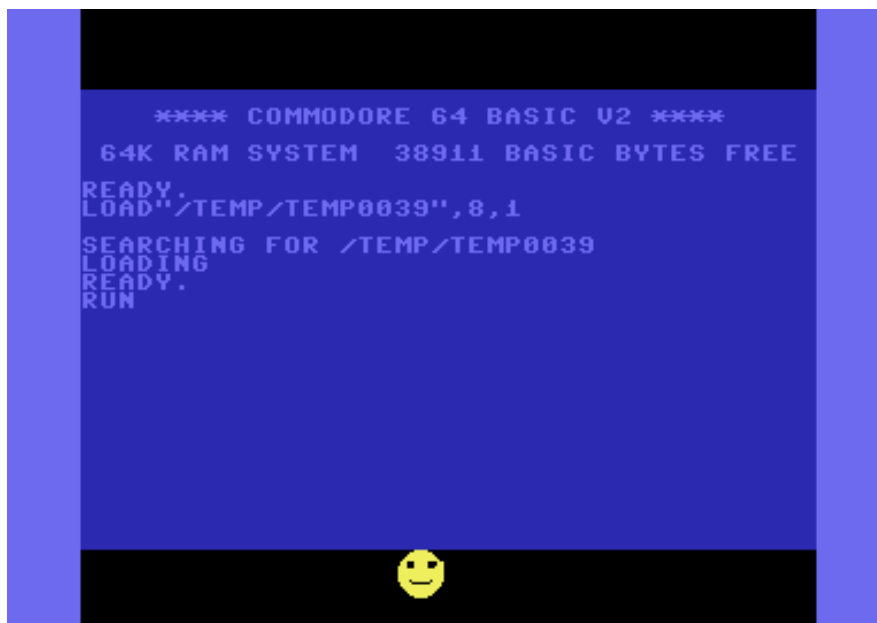
And a surprise, which measurement confirms: **there is no horizontal border left at all**, neither at the bottom nor at the top. I sampled the central column of the capture — 272 lines, a single color, not one pixel of frame — whereas a normal capture shows 35 lines of border at the top and 37 at the bottom.

Why the top as well? Reread the two rules. The bottom comparison **sets** the flip-flop to 1; the top comparison **resets** it to 0. We prevented the setting to 1; so there is nothing left to reset to 0 at the top of the next image. The switch stayed at 0 from one end to the other. We wanted to open a door, we opened the whole corridor — and that is logical, not magic.

The ghost byte

One last curiosity, and it is worth the detour. Remove the `lda #0 / sta $3fff` line from our program — or better, replace the `#0` with `#$ff` to force the phenomenon.

What you observe:



Real capture, with \$3fff deliberately set to \$ff. The zones we have just opened are black: the VIC is displaying something there, and that something comes from a single locker.

Remember the idle state from chapter 7: when the VIC has no line of text to display, it does not stop for all that — it keeps reading, always at the same address, \$3fff. And it treats what it finds there as pixels: every bit at 1 becomes a black pixel, every bit at 0 lets the background color show.

In the zones we have just opened, we are therefore not looking at “nothing”. We are looking at the contents of a single memory locker, repeated endlessly, eight pixels by eight pixels. Write zero into it, and the emptiness becomes clean again. Demo programmers have known this since forever: the first line of any code that opens a border is almost always the one that cleans \$3fff.

Under the hood — the two border flip-flops (main and vertical), their four comparators, the six exact rules that operate them, and the values for each mode combination are specified in *Au cœur du métal* — *Commodore 64 & Ultimate 64*, chapter 1, §11 (“Border unit”). The idle state and the role of \$3fff are in §7 of the same chapter.

Next chapter

We dodged a comparison. We prevented a Bad Line. What remains is to do the opposite: **cause** one, on every line, to force the VIC to re-read its colors 200 times per frame instead of 25. It is the most famous trick of the machine, and the last of this book.

Chapter 9 — The Grand Finale: All the Colors at Once

The question

The Commodore 64 has a stubborn reputation: its colors are crude. In its simplest graphics mode, the screen is cut into cells of 8×8 pixels, and **each cell can only show two colors**. Two colors for 64 pixels: that is what gives this machine's pictures their mosaic look. (There is a multicolor mode that allows four, but at the price of half the horizontal sharpness: the compromise moves the problem, it does not solve it.) And yet. Open any gallery of C64 graphics and you will find portraits with impossible gradients, skies shaded line by line. How?

The answer is the machine's most famous trick, and you already have everything you need to understand it: all it does is **cause** what chapter 7 worked so hard to prevent.

What graphics mode really is

A clarification first, because it is counter-intuitive. In graphics mode, the video matrix from chapter 5 — those 1000 lockers that each held one character — no longer holds characters. It holds **colors**: in each byte, the four high bits give the color of the cell's lit pixels, the four low bits that of the unlit pixels. The pixels themselves, the 8000 bytes that say which dot is lit, live elsewhere.

Note the consequence: colors are not read along with the pixels. They are read along with the matrix — that is, **only once every eight lines**, during the Bad Line. That is the exact origin of the limitation: it is not that the VIC refuses more than two colors per cell, it is that it only *asks* for their color once per text row.

You can already see where we are going.

The idea: a Bad Line on every line

If the VIC rereads its colors on each Bad Line, and if we know how to maneuver the vertical scroll value that triggers them — we did it in chapter 7, but to **prevent** them — then let's turn the gesture around and force one **on every raster line**. The VIC will reread its colors 200 times per frame instead of 25. Each strip 8 pixels wide and 1 pixel tall can have its own colors.

This is **FLI**, for *Flexible Line Interpretation*, invented by demo programmers in the late 1980s. It is expensive — we will see how much — but it completely changes what the machine can show.

Two obstacles stand in the way, and the documentation names both of them.

First obstacle: the VIC always rereads the same addresses. When a Bad Line occurs before the end of the current text row, the VIC does not advance its bookmark — so it rereads exactly the same 40 cells. We would have colors reread 200 times, but identical ones! The fix is brutal and elegant: we do not change the data, we change **the place where the VIC goes to fetch it**. Remember chapter 5: `$d018` says *where* the matrix is. So we prepare **eight matrices** in memory, and we switch between them on every line. Eight are enough, because, counting lines from the first Bad Line (raster line 48), line number N reads row $N/8$ of matrix number $N \bmod 8$ — the eight lines of one row of cells draw from eight different matrices.

Second obstacle: there is one precise moment to act. The write that creates the Bad Line must not arrive before **cycle 14** of the line. Any earlier, and the VIC resets its pixel-row counter: it would redisplay the same row of pixels forever. You will see this accident with your own eyes in a moment — I ran into it twice.

Three attempts, and what each one teaches

I could give you the program that works. It is better that I tell you how I found it: the two failures are more instructive than the success.

First attempt. A counting loop: 200 lines to paint, a counter that counts down, and two tables read at the counter's index. Twenty-one cycles per pass. Result: the first thirty-two lines are magnificent, then the screen turns into repetitive mush — the exact signature of the pixel counter being reset. The loop had drifted.

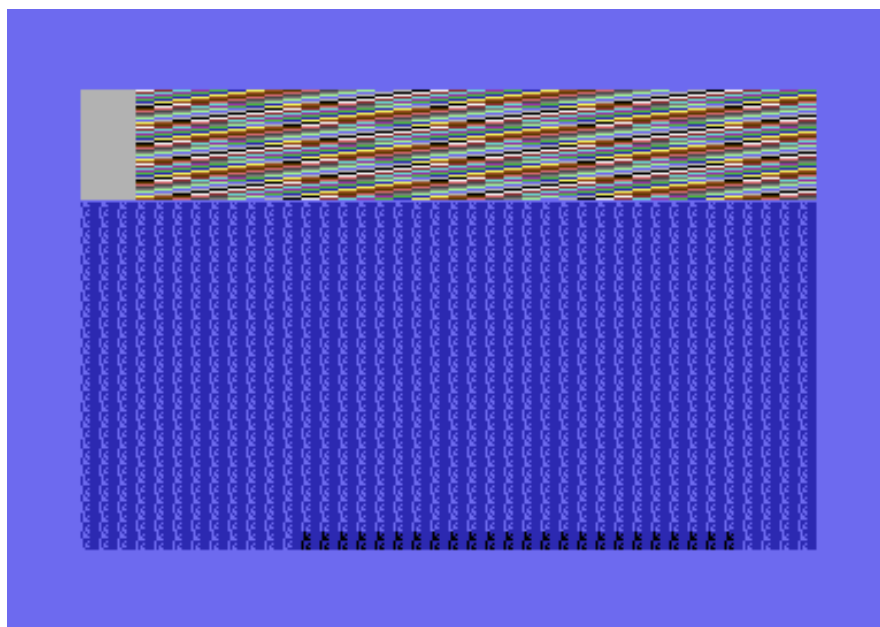
Second attempt. Let's calculate better. A line lasts 63 cycles; on a Bad Line, the reference table gives 40 cycles requisitioned, hence 23 left to the processor. You may remember chapter 6, where the measurement gave rather 41 or 43: both are true, because this theft is **not constant**. Hold on to that sentence, this whole chapter follows from it. Let's take the table's 23 cycles for now. Our loop consumed 21: two cycles ahead on every line, and the write ended up landing before the famous cycle 14. So we must *slow down* the loop by two cycles — that is the job of `nop`, the instruction that does nothing for two cycles, the programmer's shim in the C64 world. It is the fourteenth 6510 instruction this book mentions, and the only one it does not teach: you will see it only in this program, the one that does not work. Exactly twenty-three cycles. Result: the same collapse, only pushed back — forty-eight correct lines instead of thirty-two. I had bought sixteen lines, not a solution.

This second failure is the most interesting in the book, because it reveals a **reasoning error**, not a calculation error. My counter assumed "one loop pass = one screen line." That assumption is *unverifiable from the inside*: nothing in my program asked the machine where the beam really was. The counter and reality told two different stories, and nothing brought them together.

Here is the faulty loop — count it, it really does take its 23 cycles:

```
sync    lda $d012
        cmp #$33          ; we enter the display window
        bne sync
        ldx #200           ; 200 lines to paint... or so we think
rate    lda t11x,x         ; 4 : "the scroll value of the line I THINK I am painting"
        sta $d011          ; 4
        lda t18x,x         ; 4
        sta $d018          ; 4
        nop                ; 2 : the shim
        dex                ; 2
        bne rate           ; 3 -> 23 cycles
```

These two tables `t11x` and `t18x` are those of the final version, shifted by one; the complete program is in the supplied file `flirate.a`, if you want to reproduce the failure. And here is what it produces on screen — because you have to see it to understand what follows:



Real capture of the second attempt. The first 48 lines are correct, then the pattern repeats forever: the VIC redisplay the same row of pixels, because our write slipped from the right side to the wrong side of cycle 14.

Third attempt — the one that works. I threw out the counter. On every pass, the loop asks the VIC where it is, and gives it what *this* line demands:

```

fli    ldx $d012          ; 4 : the signature pattern from chapter 3
        lda t11,x          ; 4 : the scroll value that FORCES the Bad Line here
        sta $d011          ; 4
        lda t18,x          ; 4 : this line's matrix
        sta $d018          ; 4
        jmp fli            ; 3 -> exactly 23 cycles

```

You know these six lines: it is the signature pattern from chapter 3, `ldx $d012` followed by an `lda table,x`, exactly like the trailer in chapter 0. But its effect is now entirely different: the loop is **self-correcting**. If a shift occurs, the next pass reads the true line and writes the true value. There is no more drift possible, because there is nothing left to drift — no internal state, no assumption, just a question put to the machine, 15,000 times a second.

And notice why this form holds up when the other two gave way: since the Bad Line's theft is not constant, **no fixed-rate loop can follow the beam for long** — 21 or 23 cycles, drift was only a matter of patience. A loop that asks for the line again on every pass does not care about the irregularity.

And the best part: **the thief from chapter 4 has become our metronome**. Every Bad Line we cause freezes the processor until the same cycle of the line; it therefore always resumes at the same place. This freeze that we discovered as a nuisance is what keeps the whole thing in rhythm, without interrupts, without clever synchronization. The machine steals our time and, in doing so, tells us what time it is.

Where the VIC goes to fetch all this

Three values in the listing that follows deserve to be deciphered, otherwise you would take them on faith — and this book refuses that.

The bank. The VIC has only fourteen address lines: it sees only 16 kilobytes at a time, a **bank**, chosen in a locker of the CIA (`$dd00`) — yes, the doorman from chapter 0, whom we were supposed to leave alone: he holds this key too, and it is the only time we will disturb him. We need room for eight matrices and a bitmap; so we move its gaze to the bank `$4000–$7fff`. A trap waits there: the two bits that choose the bank are **wired backwards** — writing `%10` selects bank no. 1. And a cascading consequence, which is a good question to ask yourself: the ghost byte from chapter 8 lived at `$3fff`, the last address *seen by the VIC*. In bank 1, that same seen address becomes `$7fff` in real memory. That is therefore where our zero must be written — I had started by getting it wrong.

\$30 in \$d011. The locker with eight switches (appendix, reference page): here the **graphics** mode switch is on, the display is on, the window is set to **24 rows** — and the bottom three bits are left to the vertical scroll value, which our tables vary. Why 24 and not 25? The answer is at the end of the chapter, and it is worth the detour.

<< 4 and | 8 in \$d018. `<<` is an operation of the assembler, not an instruction: `x << 4` shifts the bits four places to the left, which puts a number from 0 to 15 into the top four bits of the byte. It is exactly the “multiplied by 16” from chapter 5, written differently. And it is precisely these four bits that tell the VIC **where** the matrix is: writing the matrix number there is enough to change it. Two nuances compared to chapter 5, where we did the same thing: this number is now counted **from the start of the bank** (matrix no. 1 is at `$4400`, not `$0400`), and the bottom three bits, which designated the character generator in text mode, designate the **bitmap** in graphics mode. The `| 8` turns on the one that places it in the middle of the bank, at `$6000`.

The experiment

```
!to "fli.prg", cbm

* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                    ; silence

    ; --- the VIC looks at bank 1 ($4000-$7fff) ---
    lda $dd00
    and #%11111100        ; the two bank bits are INVERTED:
    ora #%00000010        ; %10 -> bank 1
    sta $dd00

    lda #0                ; the ghost byte from chapter 8... but in
    sta $7fff              ; bank 1 it lives at $7fff, not $3fff!

; --- THE loop: 23 cycles, indexed by the raster line itself ---
fli    ldx $d012
        lda t11,x
        sta $d011
        lda t18,x
        sta $d018
        jmp fli

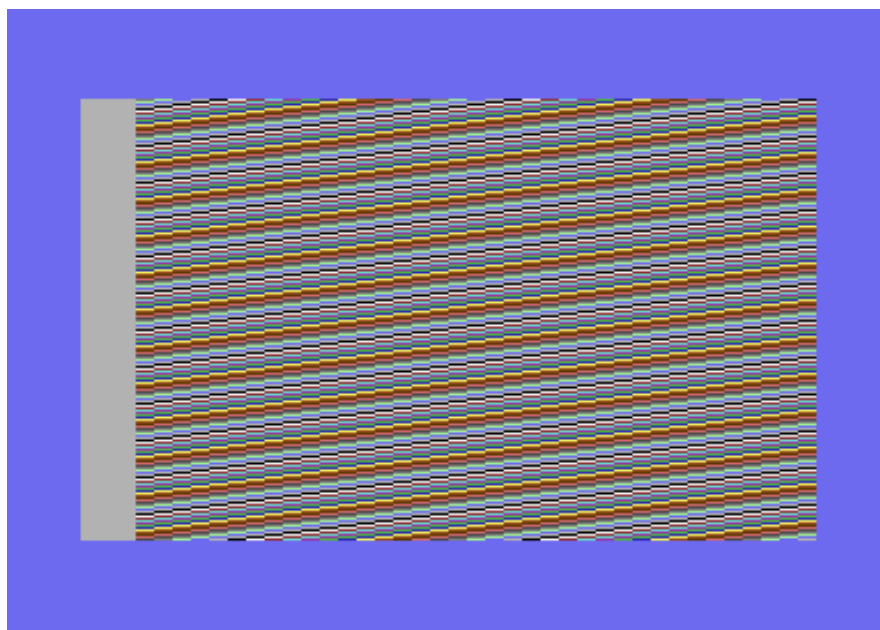
; --- the two tables, indexed by the LINE NUMBER ---
!align 255, 0
t11 !for L, 0, 255 {
    !byte $30 | (L & 7)    ; bitmap + screen + 24 LINES + scroll = L&7
}
!align 255, 0
t18 !for L, 0, 255 {
    !byte ((L & 7) << 4) | 8 ; matrix no. L&7, bitmap at $6000
}

; --- the eight video matrices ($4000, $4400, ... $5c00) ---
!for m, 0, 7 {
    * = $4000 + m * $400
    !for r, 0, 24 {
        !for c, 0, 39 {
            !byte (((r * 8 + m) + c) & 15) << 4)
        }
    }
    ; 1000 useful bytes, but a matrix takes up 1024: in the very last
    ; lines of the picture, the VIC's counter goes past 1000 and reads
    ; those 24 bytes. So we color them instead of leaving them at zero.
    !for c, 0, 23 {
        !byte (((25 * 8 + m) + c) & 15) << 4)
    }
}

; --- the bitmap ($6000): ALL pixels lit ---
; each cell therefore takes the "foreground" color from its matrix:
; we look ONLY at the colors.
* = $6000
!fill 8192, $ff          ; 1024 cells and not 1000, for the same reason
```

The useful program is **thirteen lines** long. Everything else is data, written by the assembler before the machine even starts: 8 matrices of 1024 bytes and 8192 bytes of pixels. The principle of chapter 3 pushed to its conclusion — *everything that can be computed in advance costs nothing at display time*.

What you observe:



Real capture. One hundred ninety-two lines, each with its own colors — what the Commodore 64's spec sheet declares impossible.

I checked on the machine rather than ask you to take my word: **all 192 lines** of the window each display 16 different colors, and two neighboring lines do not have the same ones. Where normal graphics mode would give 24 rows of colors, we have 192.

The scar

Look at the vertical gray band, at the far left. It is not decorative, and you cannot get rid of it.

Its cause has clockwork precision. Our write creates the Bad Line at cycle 14; the VIC wants its first cell at cycle 15. Now, for the VIC to actually take the bus at a given cycle, it must have raised its hand **three cycles earlier** — that is a delay wired into the silicon, with no waiver possible. Three of the forty cells therefore arrive too late: the VIC, doors closed, reads `$ff` instead of your colors. Three cells, eight pixels each: **24 pixels**. I measured the band in the capture — it is exactly 24 pixels wide.

The reference documentation, after explaining this mechanism, concludes with a laconic sentence: *there is no way around that*. There is no workaround. Every FLI picture in the history of the Commodore 64 bears this scar; artists got into the habit of hiding it under a black column, or of working around it. It is, literally, the trademark of the trick.

Why 24 rows, and not 25

It remains to honor the promise left along the way. Our program sets the window to 24 rows, whereas the C64 offers 25: we deprive ourselves of eight lines of picture. Why?

Because the trick has a **boundary**, and it is written in black and white in the definition of the Bad Line: it can only exist between raster lines `$30` and `$f7` — 48 and 247. Now, the 25-row window goes down to line 250. For its last three lines, no matter what we write into `$d011`, **the VIC would refuse**: no Bad Line is possible there. Deprived of new colors, it enters its idle state — and in graphics mode, the idle state displays **black**.

I know because I saw it: the first version of this program, with 25 rows, had a black streak two lines tall across the bottom of the picture. Not three: two. The VIC only switches to idle once the current pixel row is finished, and that row ended on line 248. The rule predicts “at most three,” the measurement gives two, and the gap itself can be explained — it is the kind of moment when you know you have understood the machine.

The fix is then obvious: **shrink the window to stay inside the permitted domain**. In 24 rows, the display runs from line 55 to line 246, entirely contained between 48 and 247. Not a single forbidden line left, not a single black streak. That is the price: eight fewer lines of picture, and it is exactly the compromise made by the FLI pictures you will find on the net.

(The bottom of this picture taught me yet another thing. A much wider black band lingered there, over the last eight lines: my eight matrices hold 1000 useful bytes but occupy 1024 lockers each, and the VIC's internal counter goes past 1000 in the very last lines of a frame. It was therefore reading the 24 padding bytes left at zero — 24 black cells, exactly the width measured on the capture. The two extra ! for loops in the listing color them, and the bitmap was extended from 1000 to 1024 cells for the same reason. A black band that can be counted in bytes — you couldn't make that up.)

And the price must be stated: at 200 Bad Lines per frame instead of 25, the processor spends nearly half its time frozen. An FLI picture costs not only memory — it costs the machine. That is why games almost never use it, and demos all the time.

Under the hood — the complete mechanism of FLI (non-incrementing of the matrix counter, switching among the eight matrices, the impossibility of switching the Color RAM, and the exact cause of the 24-pixel bug with the three-cycle delay between the two bus signals) is specified in *Au cœur du métal* — *Commodore 64 & Ultimate 64*, chapter 1, §16.3; the AFLI and IFLI variants are also described there.

What you know how to do

Let's stop, because the road deserves a look back. You started this book not knowing what a register was. You have just read — and understood — a program that forces the hardware to reread its colors two hundred times per frame, timed to the cycle against a theft of cycles, with a display defect that you can explain to the microsecond.

And you did it with **thirteen instructions**. Not one more.

Next chapter

One question remains, and it is a question of our time: this machine is forty years old, where does one still find it? And above all — because this is where the book has a confession to make — what on earth did I run the programs on whose captures you have seen?

Chapter 10 — The Same Machine in 2026

The question

Here we are at the end of the journey, and one question remains that you may have asked yourself from the very first page: **where does one find, in 2026, a Commodore 64 to check all this?**

You have read the same note under most of the images in this book: *real capture on C64 Ultimate*. That was the promise of the first page — all the screen images are real, each produced by the listing printed just above it. The time has come to say what this machine is, and above all why it does not cheat.

A machine described, not imitated

The object that produced the captures in this book is a **C64 Ultimate**: a re-release of the Commodore 64 sold under the Commodore brand, whose heart is an **FPGA reimplementation**. Its firmware, “Commodore 1.0”, is a stripped-down derivative of the Ultimate 64 firmware by Gideon Zweijtzer.

An FPGA — *field-programmable gate array* — is a circuit whose logic is not manufactured: it is **described**, and the circuit reconfigures itself to apply it. The difference from a software emulator is one of kind. An emulator is a program that *recounts* what a chip would have done. Here, there is no intermediate program: the VIC-II, the 6510 and their neighbors are re-described as circuits, with the single corridor, the two half-microseconds, the signal raised when the artist requisitions the bus. Same rules, same constraints — and, what is most revealing, **same quirks**.

For that is how you unmask a rough imitation: by its flaws. The FLI bug from the previous chapter — those twenty-four lost pixels on the left, about which the reference documentation says there is “no way around it” — is there, intact, measured on this machine. Better still: there is a display offset so fine, so intimate to the VIC’s internal workings, that this kind of behavior is reputed to vary, on real chips, from one silicon revision to another, and even with temperature. On this programmable circuit, it is **deterministic** — always the same, measurable once and for all. A machine that merely imitated “roughly” would have neither this bug nor this offset. This one has them.

A word of honesty, because that is this book’s discipline: the measurements I am about to discuss come from **a single unit**, taken in June and July 2026. They do not automatically hold for the whole series, nor for the firmwares to come. On your machine: check again.

Turbo: sixty-four times faster, and still twenty milliseconds

This machine can do something a 1982 Commodore 64 could not: **speed up its processor**. A menu setting, “CPU Speed”, is 1 MHz by default; it goes up to **48 MHz** on an Ultimate 64 and **64 MHz** on an Elite-II. On the C64 Ultimate of this book, the measurement has been made: the highest speed index, written into the locker \$d031 — a single byte, four of its bits for the speed index — gives **a constant 64 MHz** (provided a menu setting allows the program to touch it). Writing into this locker, by the way, has no effect on a real C64: it is an unused VIC register. The same program can therefore politely ask for turbo on both machines.

Now go back to the metaphor of chapter 0, the corridor cut into two halves. On an original Commodore 64, each microsecond offers **two slots** of access to the storeroom: one for the VIC, one for the processor. On this machine, the same microsecond offers **sixty-four**. And the heart beats permanently at this rate, whatever the setting: the speed index does only one thing, say **how many of these slots are granted to the processor**. Choosing “1 MHz” is therefore not slowing down a clock, it is claiming only a small share of it; choosing the maximum is taking them all.

The beam, for its part, has not changed its habits. Still 63 cycles per line, still 312 lines, still 19,656 beats, still **twenty milliseconds** — and that is precisely why the picture coming out of this machine is a true PAL picture, which your television accepts without argument. The processor got sixty-four times more working time; the VIC, not one microsecond less.

And here is the lesson of the book, which survives acceleration intact: **even at 64 MHz, when the VIC requisitions the corridor, the processor waits**. This is not a deduction: the freeze was measured on this machine, and its duration — **about 43 microseconds** per Bad Line, the figure the turbo documentation also gives — is that of chapter 4. The programmable core freezes the fast processor for as long as the

VIC holds the bus. Any technique that *forces* Bad Lines therefore pays this price in full, whatever speed is shown in the menu.

Do the math for the FLI of chapter 9, and savor it: about 200 forced Bad Lines, at 43 microseconds each, makes **8.6 milliseconds frozen in a 20-millisecond frame**. The processor loses **nearly half** of its time — at turbo as at 1 MHz. The title of this book has never been contradicted: everything is still played out in the same twenty milliseconds, and you still have to tuck your calculations away in the borders.

There is something even more beautiful. The people who measured this machine turned the constraint around: since the freeze happens **at the same point on every line**, it realigns the program with the beam on every iteration. The defect becomes the clock. No need for cycle-counted code: the machine resynchronizes by itself. You recognize the move — it is exactly that of chapters 7, 8 and 9. You observe a rule, you notice that it is regular, you use it.

What remains, to be complete, is that a switch in this same locker `$d031` lets you **disable Bad Line timing**: the processor then keeps working while the VIC holds the bus, provided its access stays internal to the machine. It is a compatibility setting, not a victory over time: the VIC always keeps priority, and any access toward the outside world — the cartridge port — stays at 1 MHz and only when the VIC has given the corridor back. Unplug this timing, and you no longer have a Commodore 64: you have a machine that resembles one. The sentence from chapter 0 holds firm to the last paragraph of the last chapter: **in a Commodore 64, time belongs to the video chip**.

The freight elevator: the REU

One last accessory, and it deserves the detour because it moves the question of the corridor.

The **REU** — *RAM Expansion Unit* — is a memory expansion that Commodore was already selling back then: 128 kilobytes on the 1700, 256 on the 1764, 512 on the 1750, expandable to several megabytes. Its peculiarity is that the processor **cannot address it**. It appears nowhere among the 65,536 lockers of chapter 0.

How is it accessed, then? By a **freight elevator**. The REU contains its own controller, which moves blocks of bytes between the C64's memory and its own — in one direction, in the other, or swapping the two — without the processor carrying anything at all. You give it an address here, an address over there, a length, you write the start order into a locker, and the block travels. Three ways of moving house, a fourth command that merely compares, eleven lockers from `$df00` to `$df0a`: that is the whole vocabulary.

The freight elevator has its temperament, and it has been measured: each trip costs about **60 microseconds of setup**. Sixty microseconds out of the twenty thousand of a frame is nothing for moving a whole picture; it is ruinous if you want to move thirty small things every frame. Few big transfers: yes. Many small ones: no. One could think one was hearing chapter 4.

Under the hood — the speed settings and their index table, the turbo control registers, the behavior of Bad Lines at high frequency, the complete table of REU registers and its four commands are specified in *Au cœur du métal — Commodore 64 & Ultimate 64*, chapter 6 (“L’Ultimate 64 : services”), §4, for the turbo, and chapter 5 (“Le REU : DMA `$df00`”) for the freight elevator. The measurements cited here — the 43 microseconds of freeze at 64 MHz, the 8.6 milliseconds of the FLI, the 60 microseconds of the REU — are in **appendix A** (“Le C64 Ultimate mesuré”, §A.1 and §A.5) and **appendix B** (§B.1), which report original readings on the unit described.

The gateway

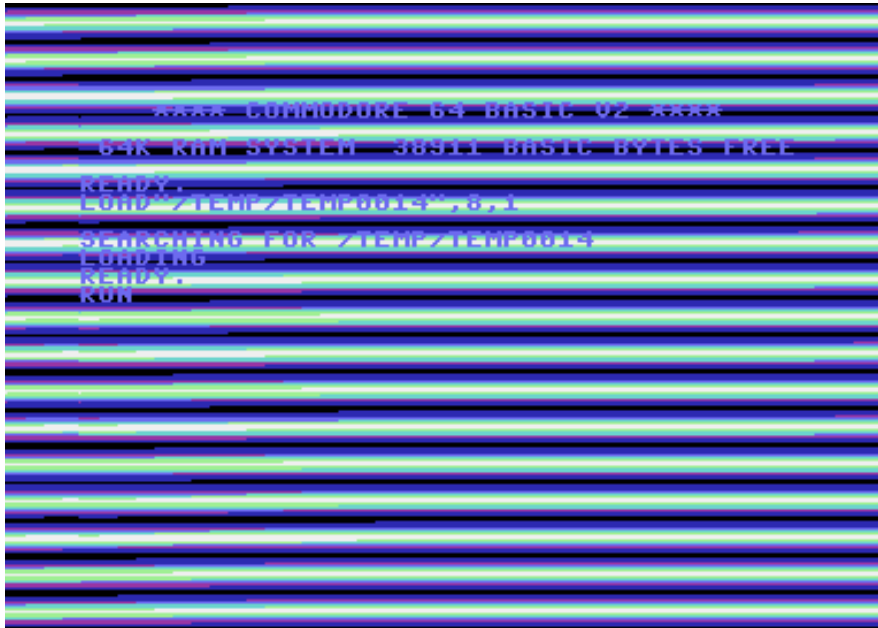
You now know how to see the twenty milliseconds. You know that a color is a write, that a ten-cycle delay reads on screen as eighty pixels, that one line in eight the processor loses its voice, and that a screen border is not a region but a switch. This book stops there: it shows the phenomena.

The day you want the **exact rules, cycle by cycle, with their sources** — the normative definition of the Bad Line, the forty-seven VIC registers one by one, the memory-access diagram of each of the 63 cycles of a line, the six rules of the border flip-flops — open *Au cœur du métal — Commodore 64 & Ultimate 64*. It is built for that, and for nothing else: every claim in it carries the anchor of the document that attests it. It does not try to teach you — it tries to prove to you. It is a different trade, and that is why two books were needed.

And if you prefer to go straight back to the sources — Bauer’s article, Mäkelä’s timing diagrams, the 6510 cycle tables — they are all gathered at the end of this book, with the address where to find them.

Coming full circle

Go back one last time to chapter 0, to the trailer you could not yet read.



Six instructions: `sei` to close the door, `ldx $d012` to ask where the beam is, `lda couleurs,x` to take from the table what was planned for that line, two `sta` to paint the border and the background, `jmp` to start again. You now read them without thinking, and you know why the waves lean slightly at the edges.

Six instructions, five different names — out of the **thirteen** in this whole book. You do not know 6502 assembler in full, and that was not the goal: you know a machine, which is much rarer. You have 19,656 beats left to fill, fifty times a second, and nobody to tell you what to do with them.

Appendix — Three Pages to Keep at Hand

This book chose to explain each thing only at the moment it becomes useful. That is good for learning, less handy for programming: three chapters in, you find yourself hunting for “the number of light red” or “which bit turns the screen off”. Here, gathered in one place, is everything the book’s programs use.

The Sixteen Colors

They are written to `$d020` (border), `$d021` (background), `$d027–$d02e` (sprites), or to Color RAM starting at `$d800`. There are no others: this palette *is* the Commodore 64.

Decimal	Hex	Color	Decimal	Hex	Color
0	<code>\$00</code>	black	8	<code>\$08</code>	orange
1	<code>\$01</code>	white	9	<code>\$09</code>	brown
2	<code>\$02</code>	red	10	<code>\$0a</code>	light red
3	<code>\$03</code>	cyan	11	<code>\$0b</code>	dark gray
4	<code>\$04</code>	purple	12	<code>\$0c</code>	gray
5	<code>\$05</code>	green	13	<code>\$0d</code>	light green
6	<code>\$06</code>	blue	14	<code>\$0e</code>	light blue
7	<code>\$07</code>	yellow	15	<code>\$0f</code>	light gray

Both notations are equivalent: `lda #2` and `lda #$02` give the same byte. The listings in this book use the decimal form for small numbers and the hexadecimal form when it makes the structure easier to read.

At power-up, the machine shows blue (6) on a light blue (14) border — those are the two values our programs put back in place when they hand control back.

The Eight Switches of `$d011`

This is the most important locker in the book: it carries chapters 4, 7, 8 and 9. A single byte, eight independent switches.

Bit	Working name	What it does
7	RST8	the ninth bit of the line number (read; see chapter 2)
6	ECM	“extended color” mode — not used in this book
5	BMM	graphics mode : 0 = text, 1 = bitmap (chapter 9)
4	DEN	display on : 0 = screen off (chapter 4)
3	RSEL	window height: 1 = 25 rows, 0 = 24 (chapter 8)
2–0	YSCROLL	the vertical scroll value , from 0 to 7 (chapters 4, 7, 9)

Hence the five values that keep coming back in the listings:

Value	In binary	What it means
<code>\$1b</code>	<code>%00011011</code>	the normal state: text, screen on, 25 rows, scroll value 3
<code>\$0b</code>	<code>%00001011</code>	the same, screen off (chapter 4)
<code>\$13</code>	<code>%00010011</code>	the normal one, but with a 24-row window (chapter 8)
<code>\$18</code>	<code>%00011000</code>	screen on, 25 rows, scroll value left to be computed (chapter 7)
<code>\$30</code>	<code>%00110000</code>	graphics mode , 24 rows, scroll value to be computed (chapter 9)
<code>\$38</code>	<code>%00111000</code>	the same, with 25 rows

Look at `$1b` and `$0b`: a single bit separates them, the display bit. That is all of chapter 4 in one line of a table.

The 63 Cycles of a Line

The book talks about “cycle 14”, “cycle 58”: here is the map. The cycles of a line are numbered **from 1 to 63** (PAL machines), number 1 starting at the instant the line counter increments — with one exception,

line 0, where that instant arrives one cycle later. At eight pixels per cycle, this numbering is also a position on the screen.

Cycles	What the VIC does
1, 3, 5, 7, 9	it fetches the data for sprites 3 to 7
11 to 15	it refreshes the dynamic memory (an electrical necessity)
12 to 14	if there is a Bad Line: it announces that it is taking the bus (the three-cycle notice)
15 to 54	it takes it for good and reads its 40 cells — this is where the processor is frozen (chapter 4)
16 to 55	it reads the pixels to display
56, 57	it reads idly (it still reads, but throws the result away)
58, 60, 62	it fetches the data for sprites 0, 1 and 2 — for the next line

Note the offset between the first two rows of this table: the VIC takes the bus three cycles before it needs it, because it has to give the processor time to finish its stroke. Those three cycles of politeness will explain the scar in chapter 9.

Two consequences that the book uses without proving them again: the write that creates a Bad Line must **not happen before cycle 14** (chapter 9 — cycle 14 itself is fine), and a sprite does not cost its cycles in the same place depending on its number (chapter 6).

The Assembler Notations

They are not instructions: the processor never sees them. They are orders given to ACME **while the program is being built**.

Notation	What it does
!to "name.prg", cbm	the name of the file to produce (cbm = with the load address at the front, the Commodore convention)
* = \$0810	"what follows is stored starting at this address". It is not a multiplication: * stands for the current address
NAME = \$32	gives a name to a number, so you only have to change it in one place
!byte 1,2,3	drops these bytes as they are into the program (data, not code)
!fill 8000, \$ff	drops the same byte 8000 times
!for i, 0, 255 { ... }	repeats the block for i = 0, 1, ... 255 — a build-time loop, which does not run in the machine
!macro name { ... } then +name	defines a piece of code and calls for it; the assembler copies it out at every call
!align 255, 0	advances to the next address whose 8 low bits are zero, that is, to the start of a 256-byte page (255 is the mask, 0 the wanted value)

And three operators, also computed at build time — they are the cousins of the and and ora instructions from chapter 6:

x & 7	keeps only the last three bits of x (because 7 is written %00000111)
a b	turns on in a the bits that are on in b
x << 4	shifts the bits of x four places to the left — so it stores a number from 0 to 15 in the top four bits of the byte

The Thirteen Instructions, and What They Cost

This whole book fits inside these thirteen. The costs are those of a PAL C64, where a screen line lasts 63 cycles.

Instruction	What it does	Cost
lda #7	load a value into A	2
lda \$d012	load the contents of a locker	4
lda table,x	load cell number X of a table	4 (5 if the table crosses a page)
ldx,ldy	same, for the X and Y registers	same costs as lda
sta \$d020	store A in a locker	4
sta table,x	store A in cell number X	5
inc \$d020	add 1 to the contents of a locker	6
dex	subtract 1 from register X	2
cmp #\$80	compare A with a value	2

Instruction	What it does	Cost
and #%11110111	turn off bits of A	2
ora #%00001000	turn on bits of A	2
bne boucle	jump if the last result was not zero (after cmp: not equal)	2 if you don't jump, 3 if you do (4 if the target is on another page)
jmp boucle	jump	3
sei	close the door to interrupts	2
rts	hand control back to whoever called us	6

The book's loops, rebuilt with this price list: the line watch is 9 cycles (lda+cmp+bne), the stopwatch 9 (inc+jmp), the gradient 19, the patience loop 20, the FLD turn 21, the FLI loop 23.

The Screen Codes Used in This Book

Careful, these are **not** the codes of the characters you type: they are drawer numbers in the character generator (chapter 5).

Code	Character	Code	Character
1 to 26	the letters A to Z, in order	32	space
48 to 57	the digits 0 to 9	81	a solid disc (the "heart" of old listings is code 83)

Add 128 to any of these codes to get it in reverse video.

What All the Programs Assume

No listing in this book touches \$d016, \$d017, \$d01b, \$d01c or \$d01d. They therefore count on the state the KERNAL installs at power-up and at every reset: \$d011 = \$1b, \$d015 = 0 (no sprite on), \$d016 = \$08, \$d017 = 0 (no vertical stretch), \$d018 = \$14, \$d01b = \$d01c = \$d01d = 0 (sprites in front of the background, a single color, no horizontal stretch), border 14, background 6, and the VIC bank on the first 16 kilobytes.

That is why the creatures of chapter 6 are 24×21 pixels and pass in front of the text without any program having asked for it. If an experiment behaves differently, reset the machine before running it again.

How to Exit a Program

Almost all the programs in this book that loop forever begin with sei, which cuts off the keyboard (the exception is the strobe light of chapter 1, written before we knew sei). In every case, once the program is off in its loop, the RUN/STOP key alone is not enough. To get control back: **RUN/STOP + RESTORE** — this combination goes through another door, which sei does not close, and puts the VIC's registers back in their normal state. On an emulator, resetting the machine works too.

Sources

This book never quotes a figure it has not checked somewhere. Here is where, and what each document contributes. Almost all of them are freely available and fit in a text file — that is one of the charms of this machine: its reference documentation was written by people who took it apart, and it has remained readable.

The Companion Volume

Au cœur du métal — *Commodore 64 & Ultimate 64* is the reference of which this book is the guided tour: every “Under the hood” box points to it by chapter and section.

Two clarifications do it justice. First, **it is not a primary source**: it was written *from* the documents listed below. It gathers them, cross-checks them, translates them, and anchors each of its claims to the one who attests it — it is a work of compilation and verification, not of discovery. When a doubt concerns a cycle, it is to the sources that you must go back, and it leads you there.

Second, **it has no teaching purpose**, and it owns that: it states the rule, it does not explain it. It is to be consulted, not read from cover to cover — a beginner would drown in it by the third page. That is precisely why the book you are holding exists. The two are complementary and do not do the same job: here the phenomena, hands in the grease and permission to be wrong; over there the exact rule, where it comes from, and nothing else.

One last bit of transparency, since we are at it. That volume was made exactly like this one: **written by an artificial intelligence**, under the direction of its human editor, then proofread by cross-checking several models and compared line by line with the original documents — and it is from that mania for checking everything that this book inherited its own, down to measuring its own screenshots.

And it is **the elder of the two**. “20 Milliseconds” was born from reading it: everything in it was right, and nobody could learn from it. The book you have just finished is the answer to that observation — the pedagogue child of an austere ancestor.

The VIC-II, and Time

- **Christian Bauer**, *The MOS 6567/6569 video controller (VIC-II) and its application in the Commodore 64*, revision of September 29, 2024. <https://www.cebix.net/VIC-Article.txt> The reference text. From it come the definition of the Bad Line (chapter 4), the six rules of the border flip-flops (chapter 8) and the mechanism of FLI (chapter 9). Mind the revision: the 2024 one corrects the vertical expansion of sprites compared with the 1996 text that is still circulating.
- **Marko Mäkelä**, *The memory accesses of the MOS 6569 VIC-II...* (known as “pal.timing”), June 3, 1994. File `pal.timing` at <http://www.zimmers.net/anonftp/pub/cbm/documents/chipdata/> The timing diagrams measured on real hardware: at which exact cycle each access takes place. It is the map of the 63 cycles in our appendix.
- **Pasi Ojala**, “Missing Cycles”, *C=Hacking* no. 3, 1992. <http://www.zimmers.net/anonftp/pub/cbm/magazines/c=hacking/> The founding article on cycle stealing (chapter 6). Its approximations have since been corrected by Mäkelä — who says so himself, and it is a fine lesson in method.
- **Linus Åkesson**, “Massively Interleaved Sprite Crunch”, 2016. <https://linusakesson.net/scene/lunatico/misc.php> How far sprite DMA can be pushed — demonstrated by a production that actually runs.

The Processor

- **John West and Marko Mäkelä**, *Documentation for the NMOS 65xx/85xx Instruction Set* (known as “64doc”), June 3, 1994. File `64doc`, same address as `pal.timing`. The cycle cost of each instruction, step by step. Every count in this book — 9 cycles, 19, 20, 21, 23 — can be checked in it.

The Chips This Book Left Alone

- **MOS Technology**, datasheet of the **6581 SID**, 12 pages (archive 6581.zip, same address).
- **Wolfgang Lorenz**, *A Software Model of the CIA6526*, version 2.15, May 1997 — the model that served as the basis for most emulators.
- **Richard Hable**, *Programming the Commodore REU* (files programming.reu and reu.registers, same address) — the freight elevator of chapter 10.

The 2026 Machine

- **Gideon Zweijtzer**, official Ultimate-64 / 1541 Ultimate-II documentation: <https://github.com/GideonZ/1541u-documentation>, made readable at <https://1541u-documentation.readthedocs.io> — the API, the video streams, turbo mode.  It describes the **Ultimate 64**; the machine behind our screenshots is a Commodore-branded **C64 Ultimate**, whose firmware is a reduced fork. The two diverge on several points: when in doubt, the hardware decides, not the document.

The Original Manual

- **Commodore**, *Commodore 64 Programmer's Reference Guide*, 1982 — the screen codes (appendix B) and the “Programming Graphics” chapter, which the tables in chapter 5 come from.
- **Sheldon Leemon**, *Mapping the Commodore 64* — the annotated map of memory, locker by locker.